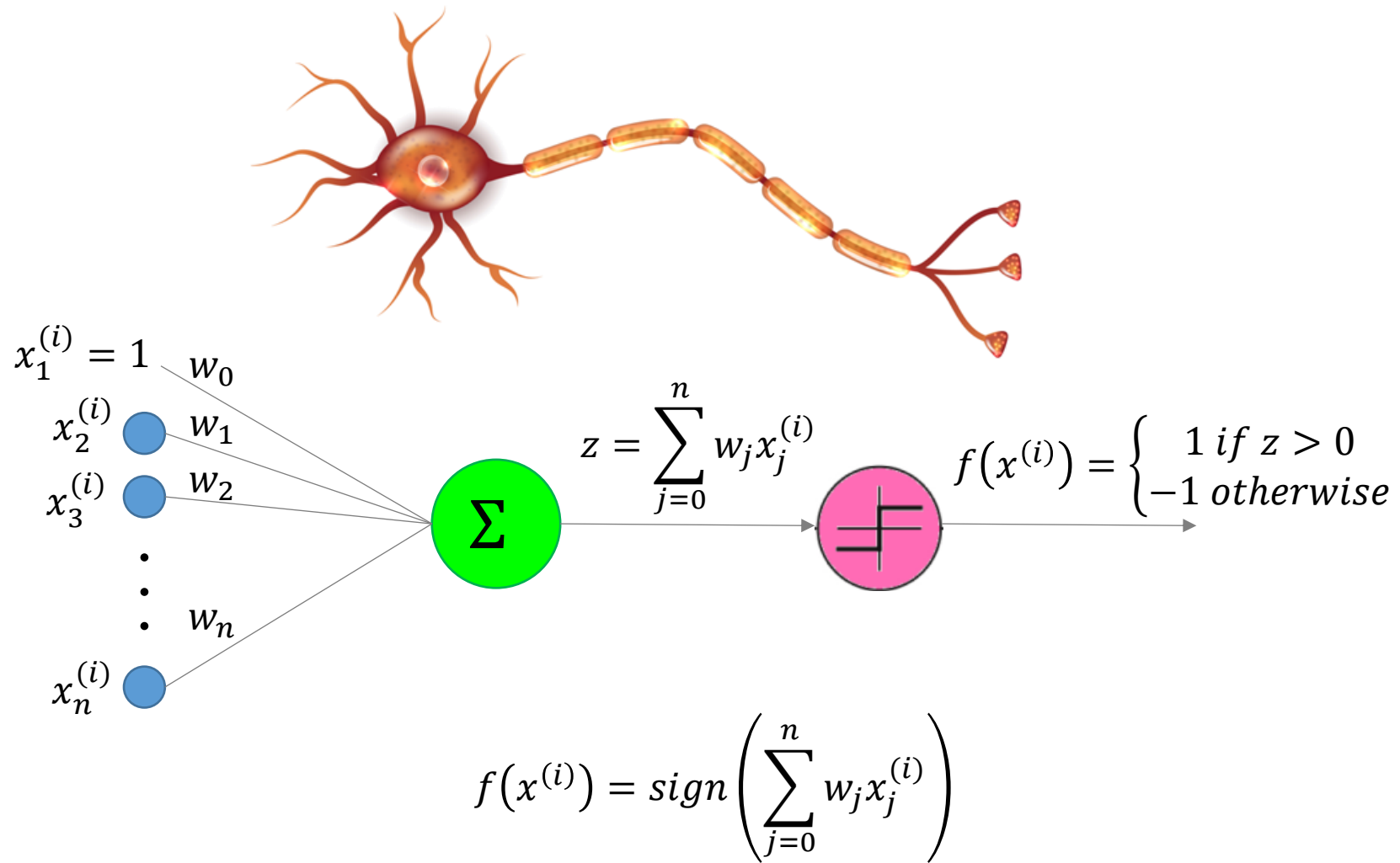


Neural Networks and Deep Learning

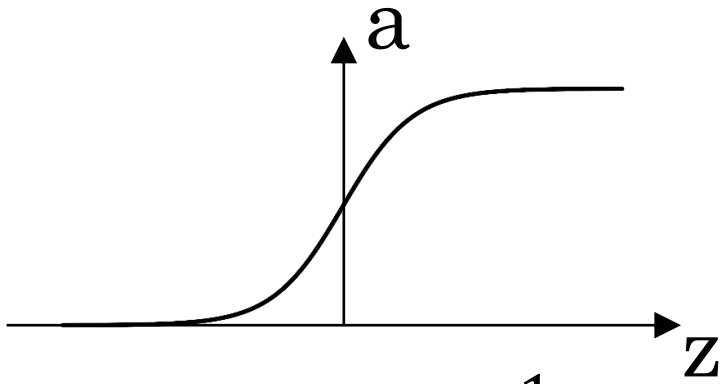
Neural Networks

- Algorithms that try to mimic how the brain functions
- NN were popular in the 90's but then lost some of its popularity
- Now back with lots of success with **deep learning** thanks to
 - data
 - computation
 - algorithms
- The first algorithm used was the Perceptron
 - Resenblatt 1959

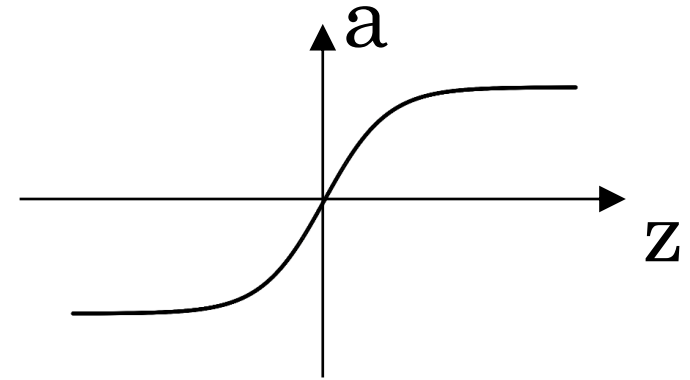
Perceptron



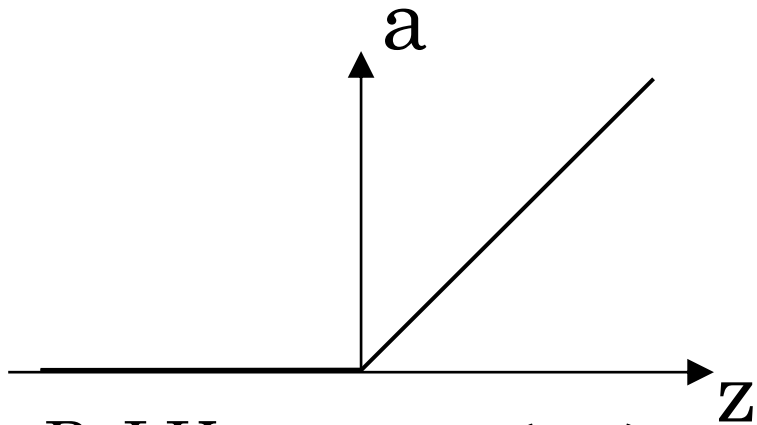
Activation Functions



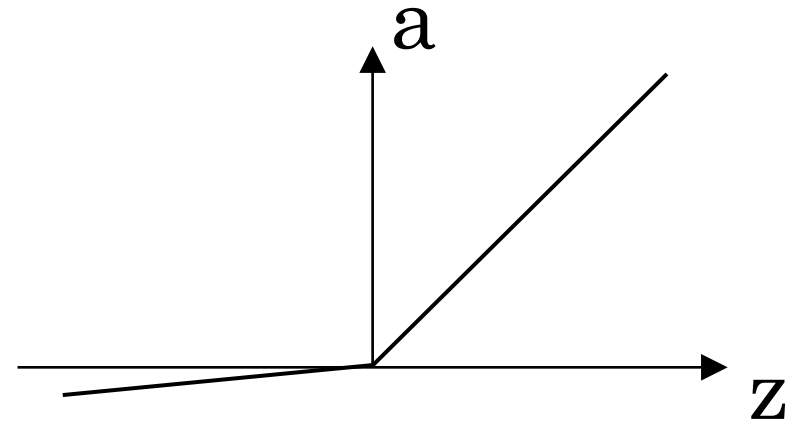
sigmoid: $a = \frac{1}{1 + e^{-z}}$



tanh: $a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

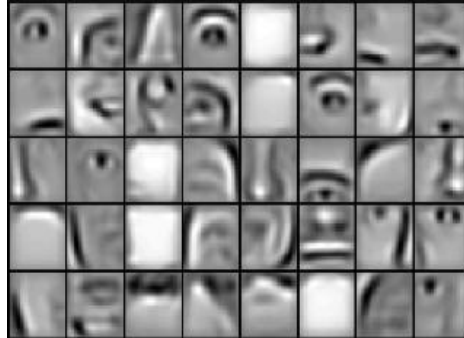
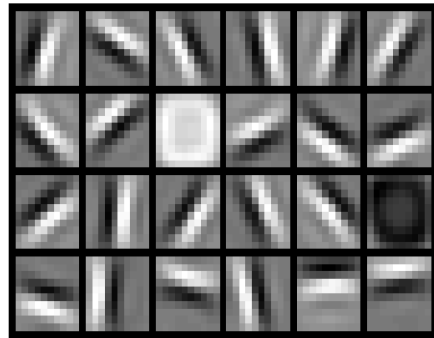
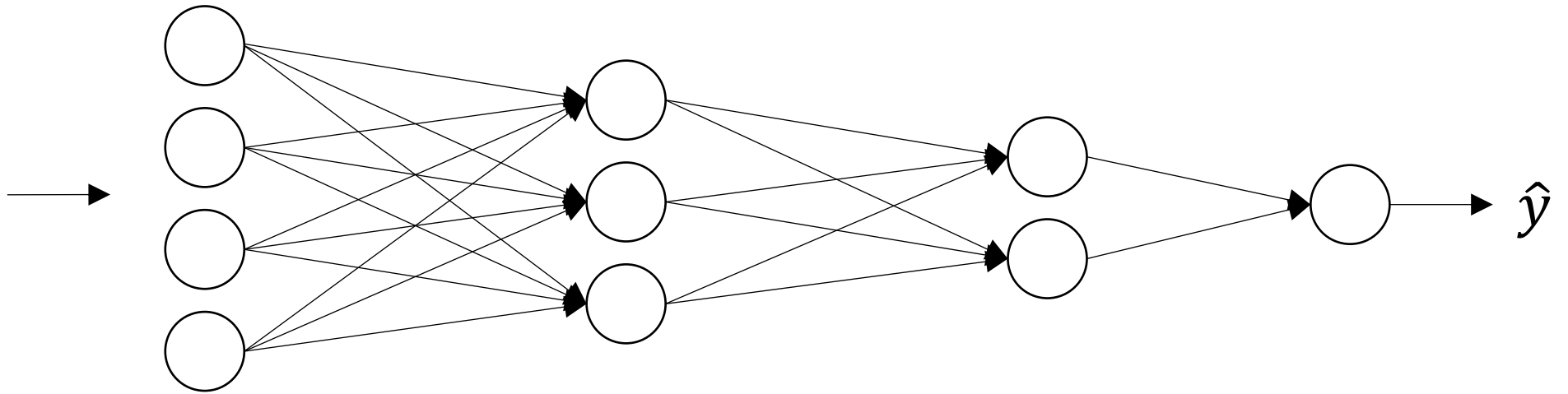


ReLU: $a = \max(0, z)$

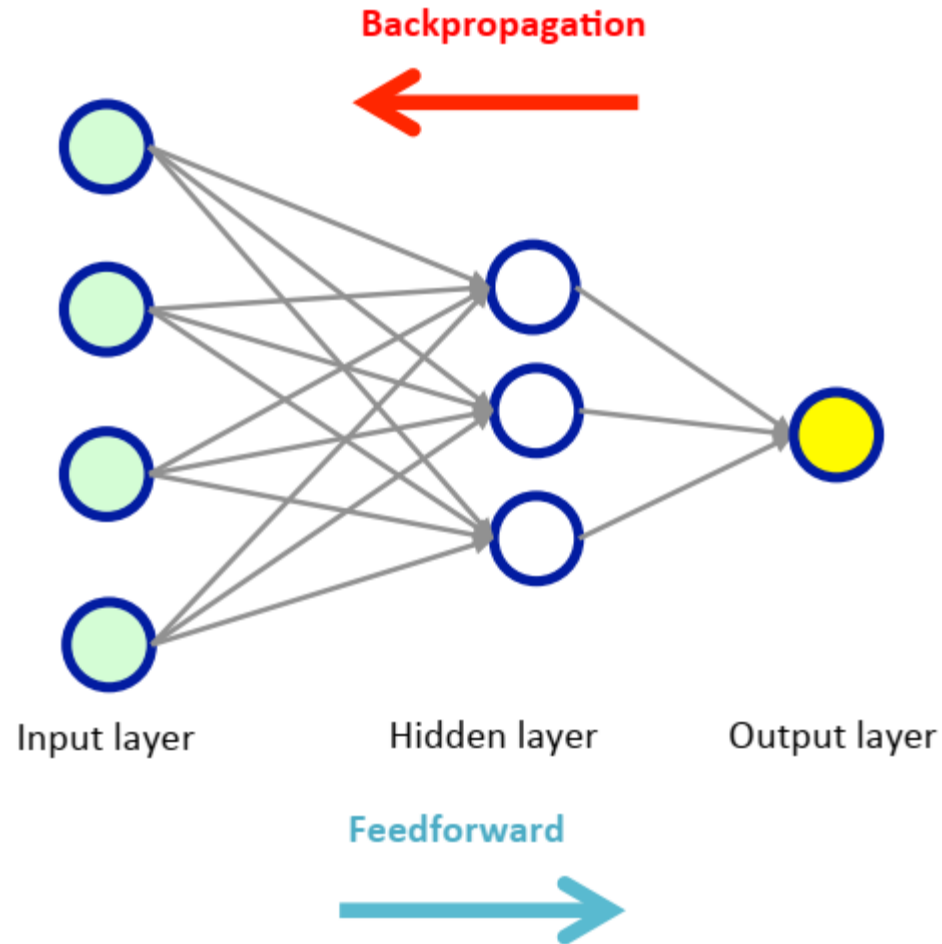


LReLU: $a = \max(0.01z, z)$

Intuition about deep representation



Feedforward – Backpropagation



- Learning the weights?
- Use gradient descent to minimize the error
 - $\mathcal{L}(y^{(i)}, \hat{y}^{(i)})$
- Backpropagation:
 - “Backward propagation of errors”

What are hyperparameters

- **Parameters:** $W^{[1]}$, $b^{[1]}$, $W^{[2]}$, $b^{[2]}$, $W^{[3]}$, $b^{[3]}$...
- Hyperparameters:
 - Learning rate
 - # of epochs/ iterations
 - # of hidden layers
 - # of hidden units
 - Choice of activation function
 - Optimization algorithm
 - Mini-batch size
 - regularization

Intuition about deep representation

- Basics: Images and Convolution
- Convolutional Neural Networks
 - Conv Layer
 - Relu Layer
 - Pooling Layer
 - FC Layer
- Training CNNs
- Designing a CNN in Keras

Convolutional Neural Networks

- Sounds like a weird combination of biology and Math with a little CS sprinkled in
- One of the most influential innovations in the field of computer vision
- 2012: **Alex Krizhevsky** wins that year's ImageNet competition (The annual Olympics of computer vision)
- Classification error dropped from **26% to 15%**

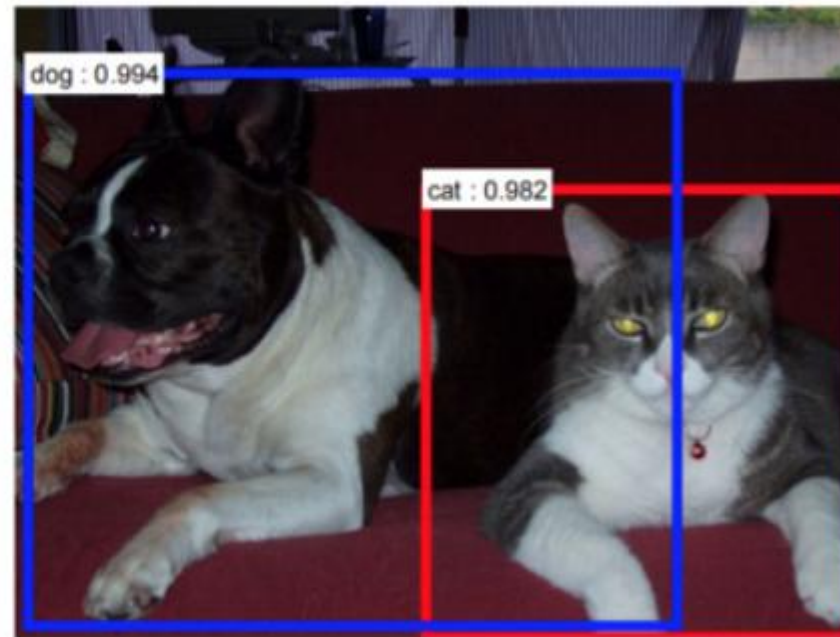
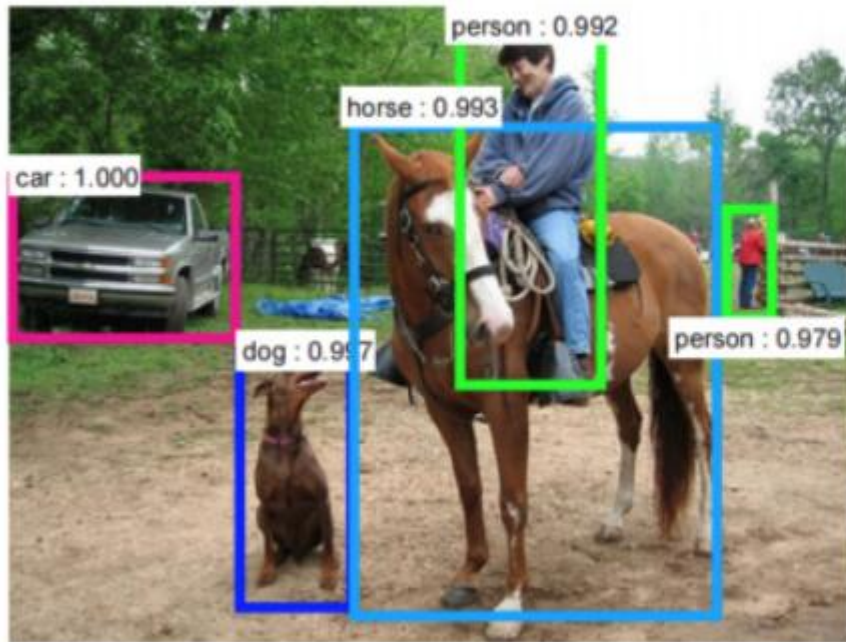
Applications of CNN

- **Facebook:** Automatic tagging algorithms
- **Google:** Photo search
- **Amazon:** Product recommendations
- **Pinterest:** Home feed personalization
- **Instagram:** Search infrastructure.



Applications of CNN

- Recognizing everyday objects



Applications of CNN

- A ConvNet is able to recognize scenes and the system is able to suggest relevant captions



a soccer player is kicking a soccer ball

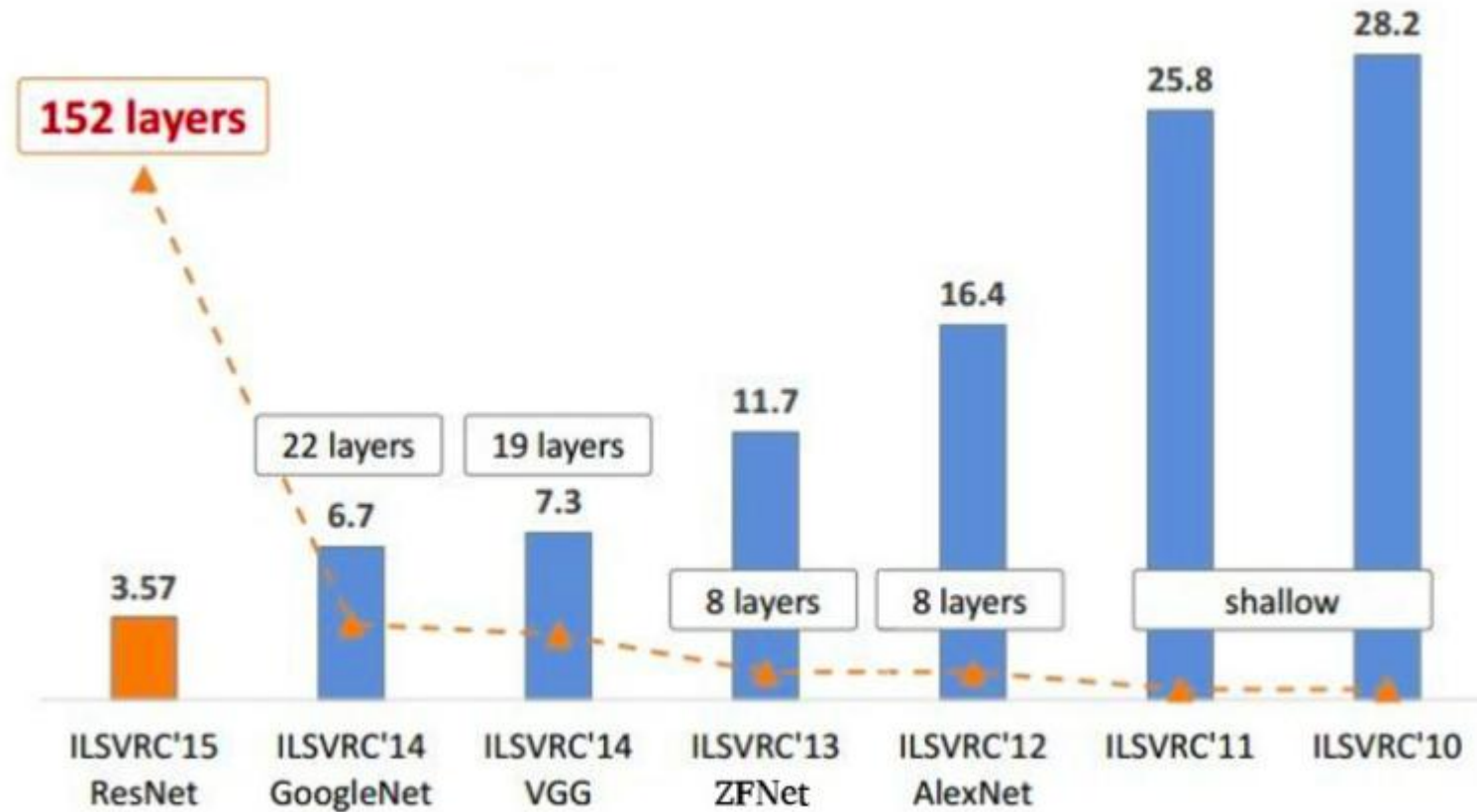


a street sign on a pole in front of a building



a couple of giraffe standing next to each other

Evolution of Depth



Deep Learning=Learning Representations/Features

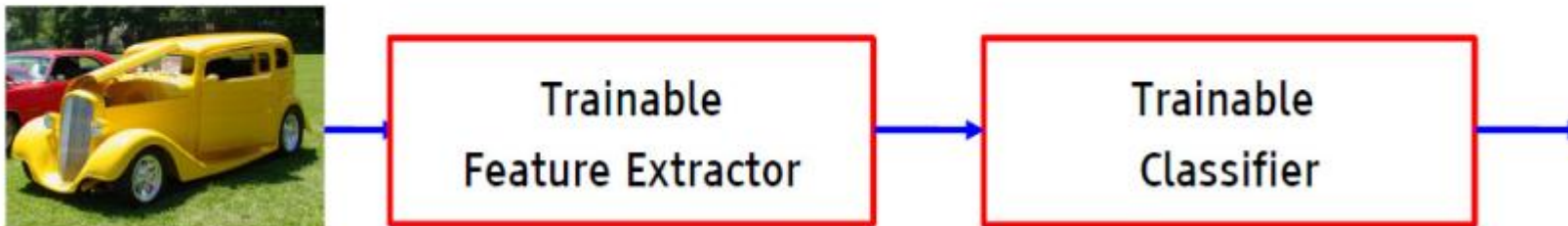
■ The traditional model of pattern recognition (since the late 50's)

- ▶ Fixed/engineered features (or fixed kernel) + trainable classifier



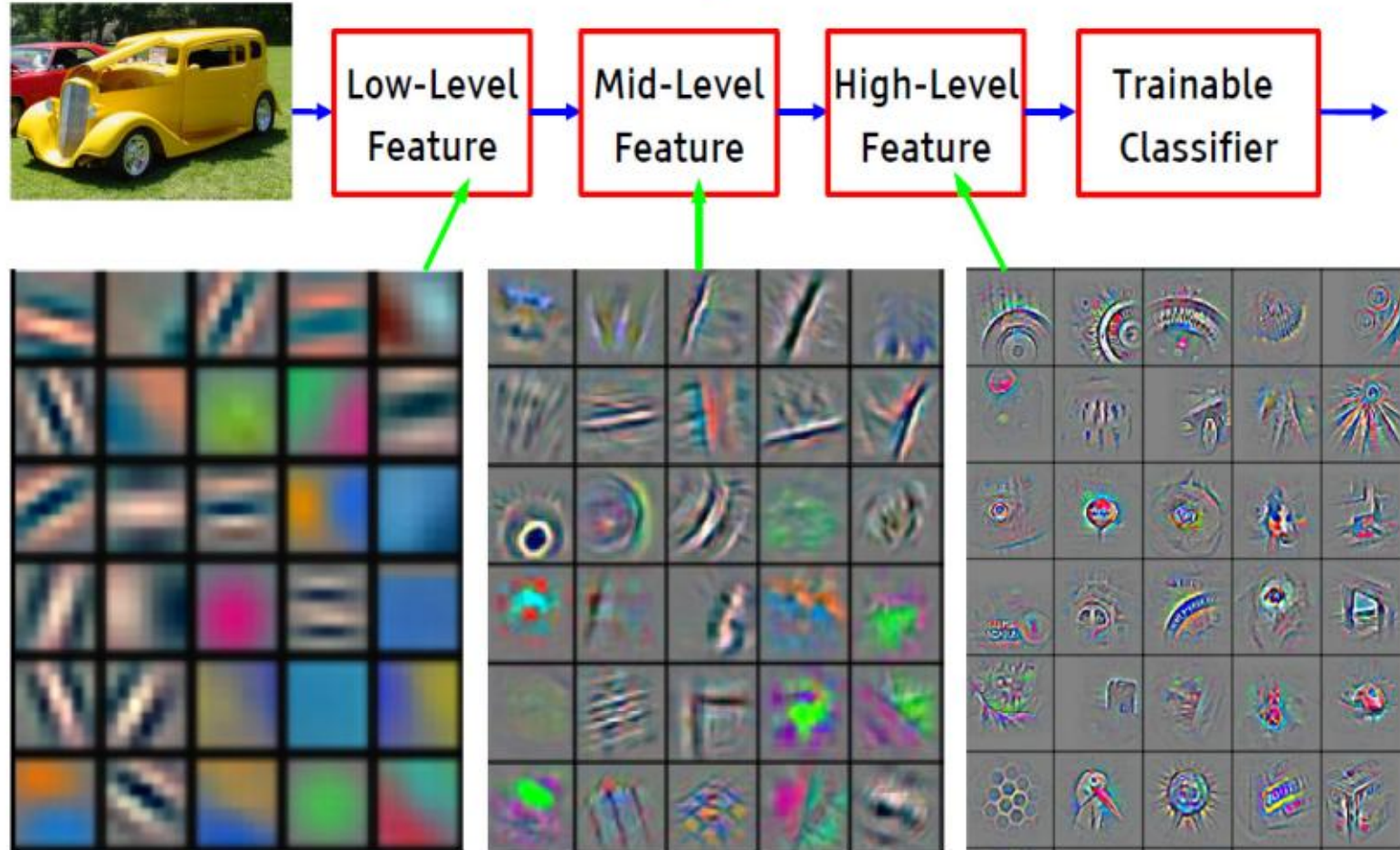
■ End-to-end learning / Feature learning / Deep learning

- ▶ Trainable features (or kernel) + trainable classifier



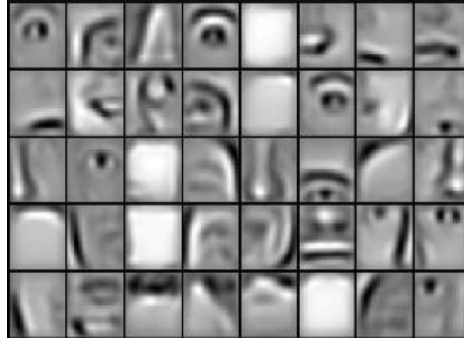
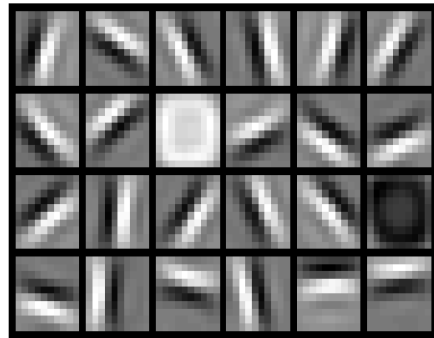
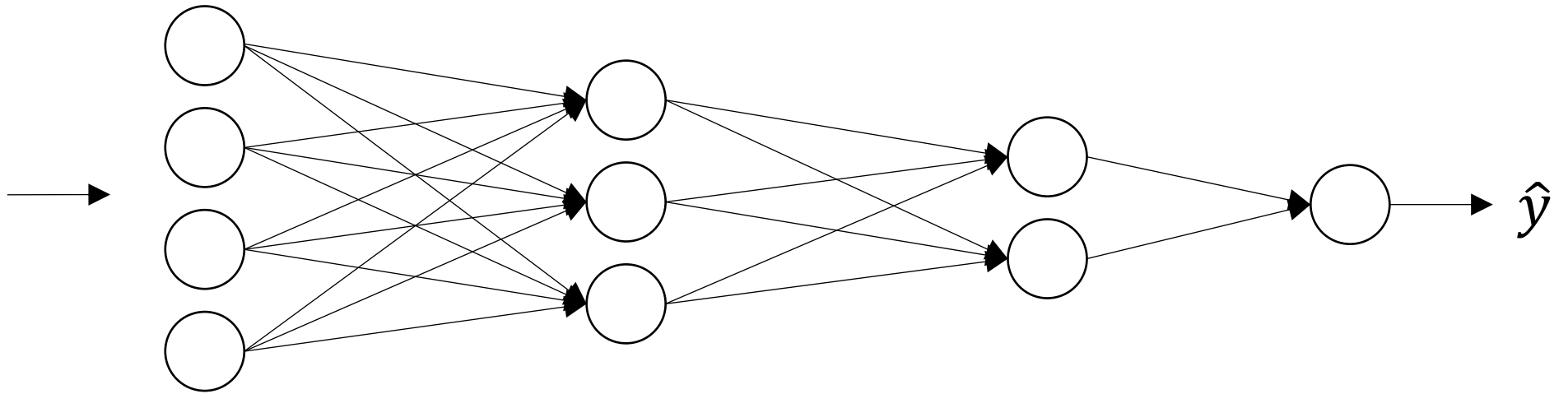
Deep Learning=Learning Representations/Features

■ It's **deep** if it has **more than one stage** of non-linear feature transformation

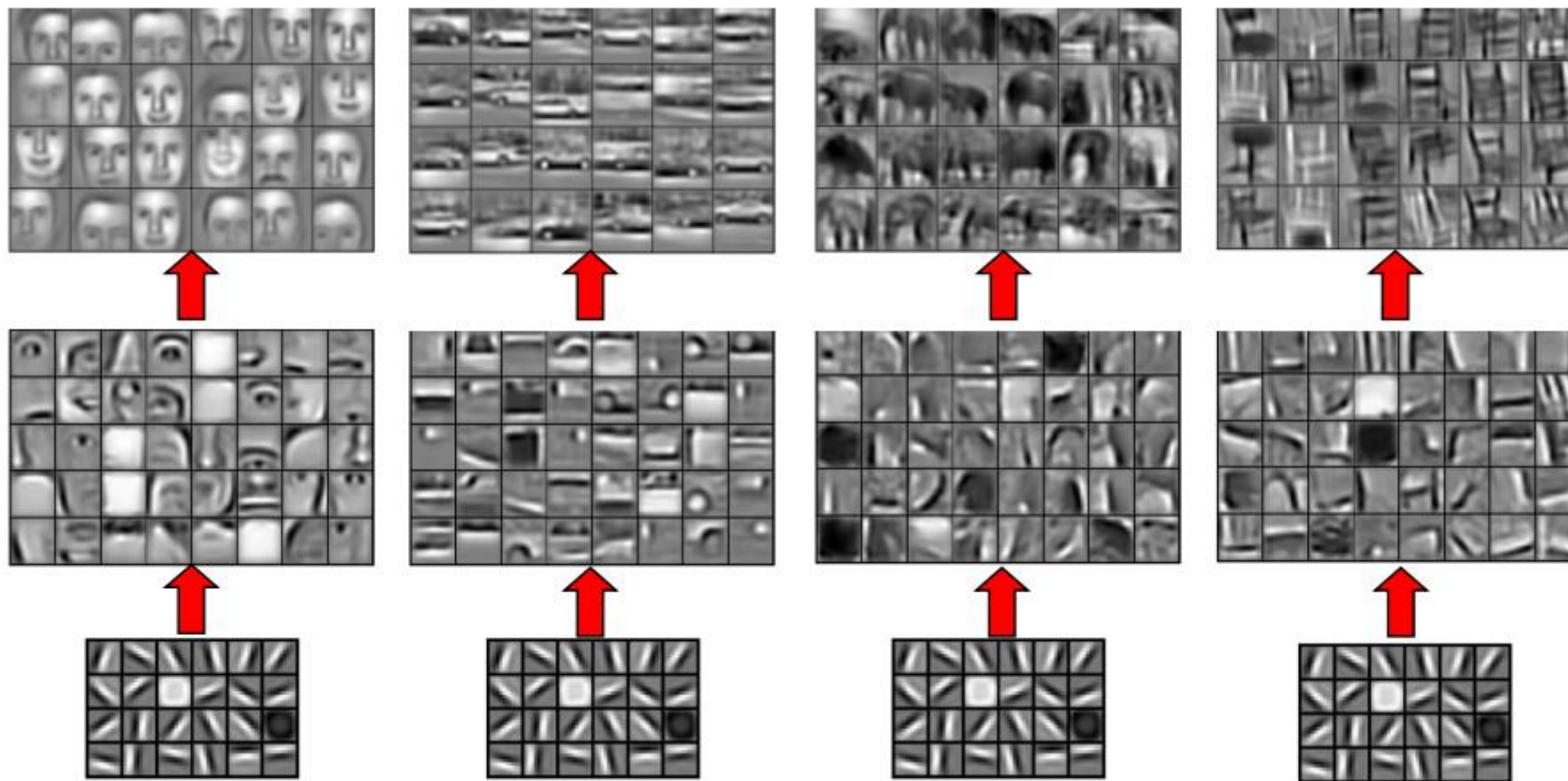


Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Intuition about deep representation



Learning of Object Parts



Recognition Task (Images)

- Humans: comes naturally and effortlessly as adults

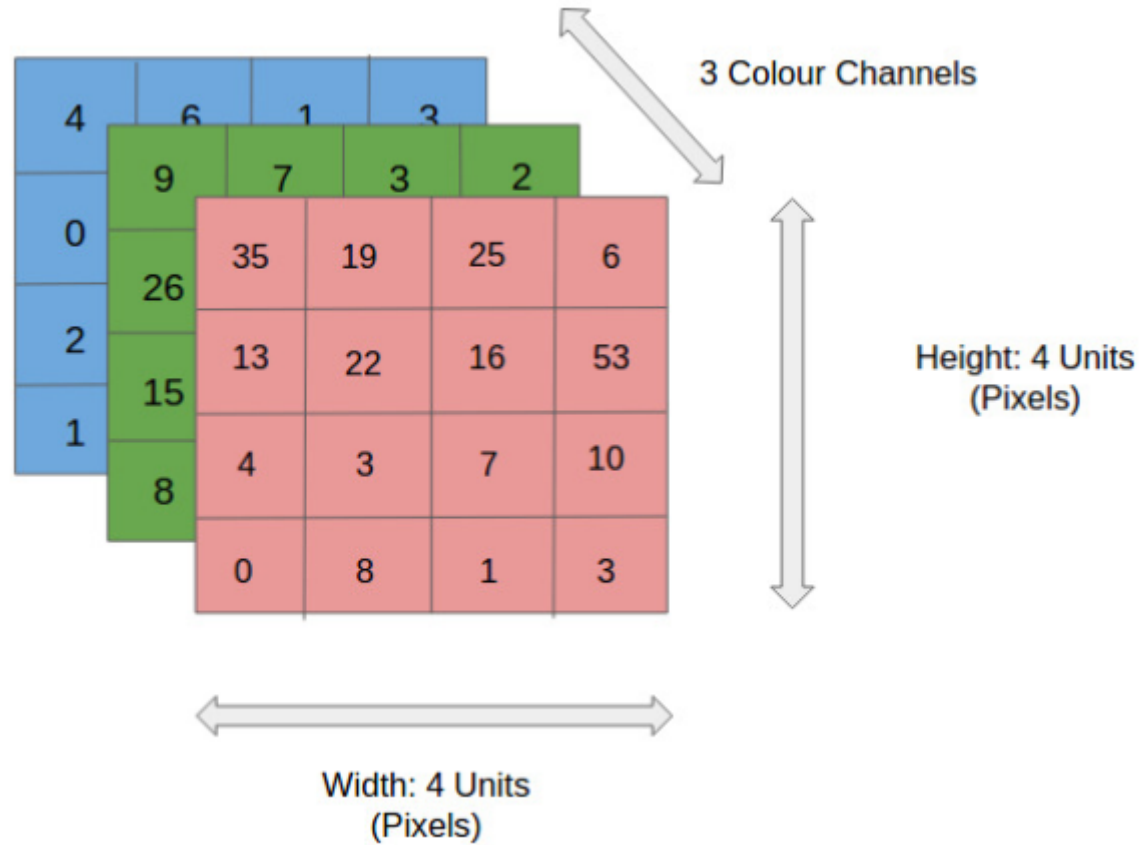


What We See

```
08 02 22 97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
49 49 99 40 17 81 18 57 60 87 17 40 98 43 69 48 04 56 62 00
81 49 31 73 55 79 14 29 93 71 40 67 53 88 30 03 49 13 36 65
52 70 95 23 04 60 11 42 69 24 68 56 01 32 56 71 37 02 36 91
22 31 16 71 51 67 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 50
32 98 81 28 64 23 67 10 26 38 40 67 59 54 70 66 18 38 64 70
67 26 20 68 02 62 12 20 95 63 94 39 63 08 40 91 66 49 94 21
24 55 58 05 66 73 99 26 97 17 78 78 94 83 14 88 34 89 63 72
21 36 23 09 75 00 76 44 20 45 35 14 00 61 33 97 34 31 33 95
78 17 53 28 22 75 31 67 15 94 03 80 04 62 16 14 09 53 56 92
16 39 05 42 96 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
86 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
19 80 81 68 05 94 47 69 28 73 92 13 86 52 17 77 04 89 55 40
04 52 08 83 97 35 99 16 07 97 57 32 16 26 26 79 33 27 98 66
88 36 68 87 57 62 20 72 03 46 33 67 46 55 12 32 63 93 53 69
04 42 16 73 38 25 39 11 24 94 72 18 08 46 29 32 60 62 76 36
20 69 36 41 72 30 23 88 34 62 99 69 82 67 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 34
01 70 54 71 83 51 54 69 16 92 33 48 61 43 52 01 89 19 67 48
```

What Computers See

Image: A Matrix of Pixel Values



Recognition Task (Images)

- Image: An array of pixel values
- Depending on the resolution and size of the image, it will see a **32 x 32 x 3** array of numbers (The 3 refers to RGB values)
- Each of these numbers is given a value from 0 to 255 which describes the pixel intensity at that point
- These numbers, while meaningless to us when we perform image classification, are the only inputs available to the computer.
- Give the computer this array of numbers and it will output numbers that describe the probability of the image being a certain class (.80 for cat, .15 for dog, .05 for bird, etc).

Convolution

- Convolution is a mathematical operation to merge two sets of information.
- In our case, Convolution is applied on the input data using a *convolution filter* to produce a *feature map*

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input

1	0	1
0	1	0
1	0	1

Filter / Kernel

Convolution

- Performed by sliding the filter over the input.
- At every location, we do element-wise matrix multiplication and sum the result. This sum goes into the feature map.

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

Input x Filter

4		

Feature Map

Convolution

- Performed by sliding the filter over the input.
- At every location, we do element-wise matrix multiplication and sum the result. This sum goes into the feature map.

1	1x1	1x0	0x1	0
0	1x0	1x1	1x0	0
0	0x1	1x0	1x1	1
0	0	1	1	0
0	1	1	0	0

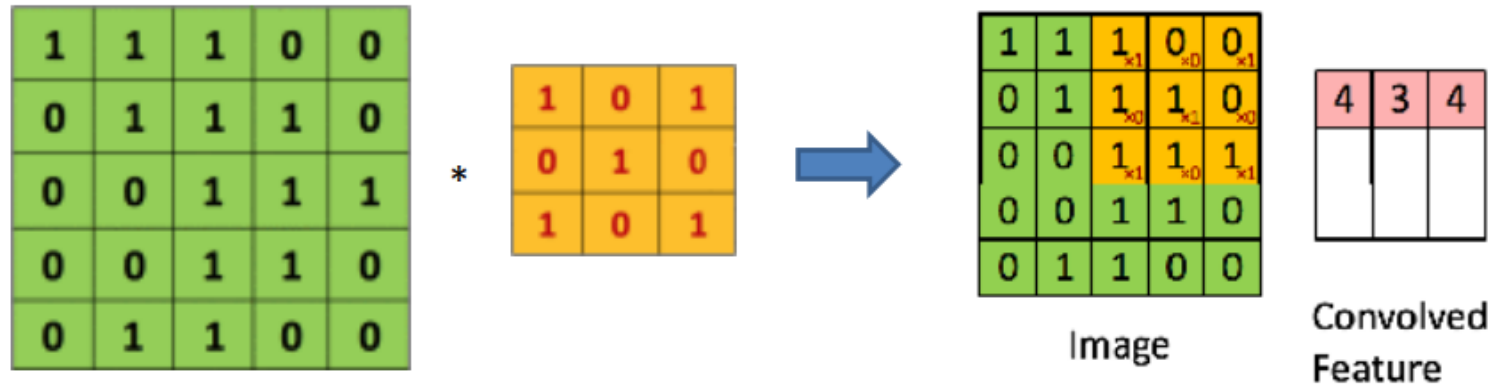
Input x Filter

4	3	

Feature Map

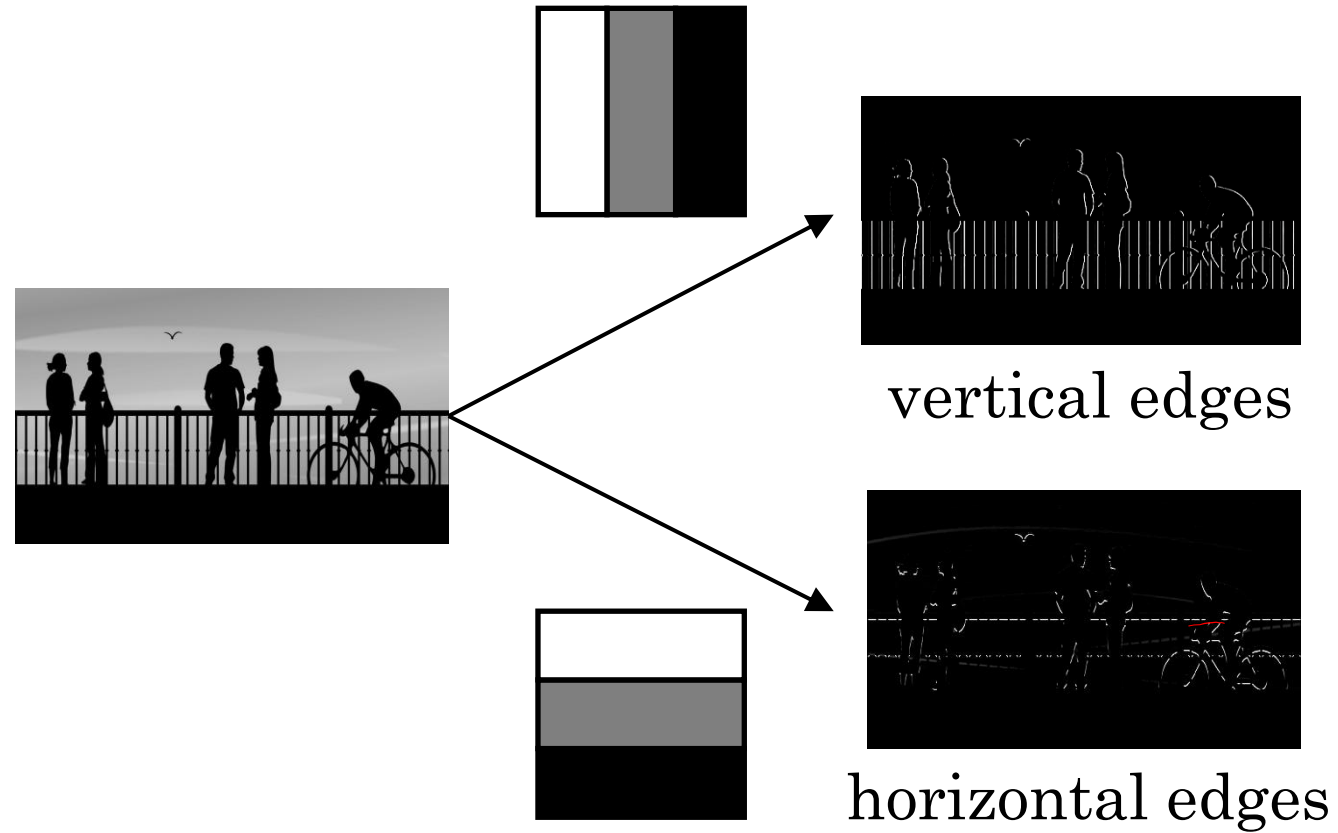
Convolution

- The primary purpose of Convolution in case of a ConvNet is to extract features from the input image



- Output of Convolution: Feature Map

Conv Layers



Conv example - vertical edge detection

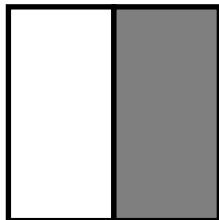
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

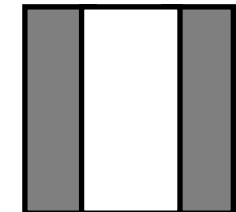
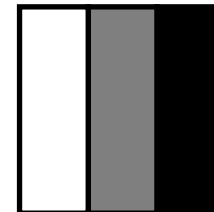
1	0	-1
1	0	-1
1	0	-1

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



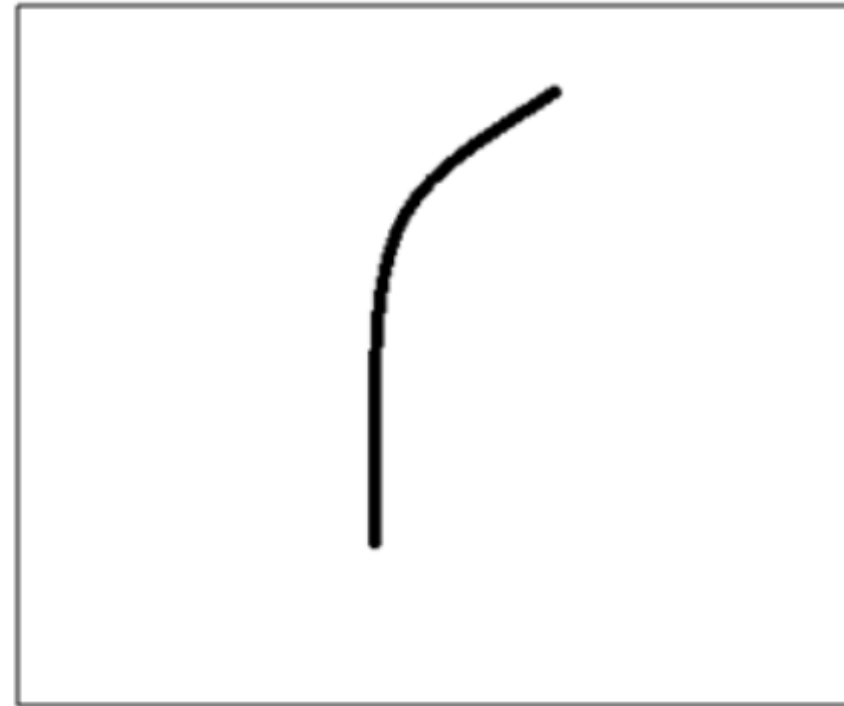
*



Convolutional Layer – High Level Perspective

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter



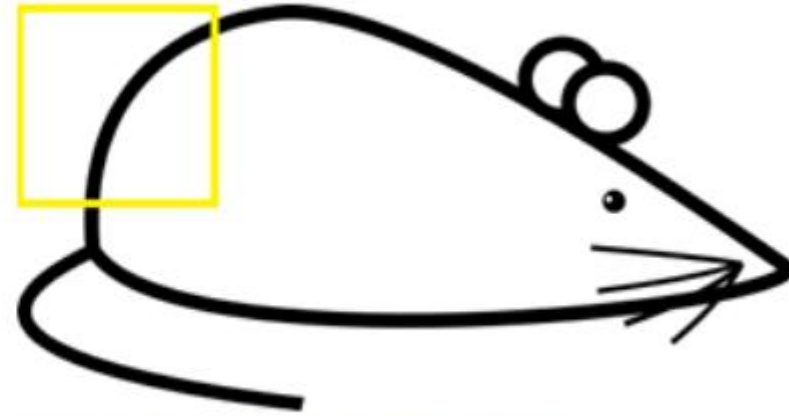
Visualization of a curve detector filter

Convolutional Layer – High Level Perspective

- If there is a shape that generally resembles the curve that this filter is representing, then all of the multiplications summed together will result in a large value



Original image



Visualization of the filter on the image

Convolutional Layer – High Level Perspective



Visualization of the receptive field

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

Pixel representation of the receptive field

*

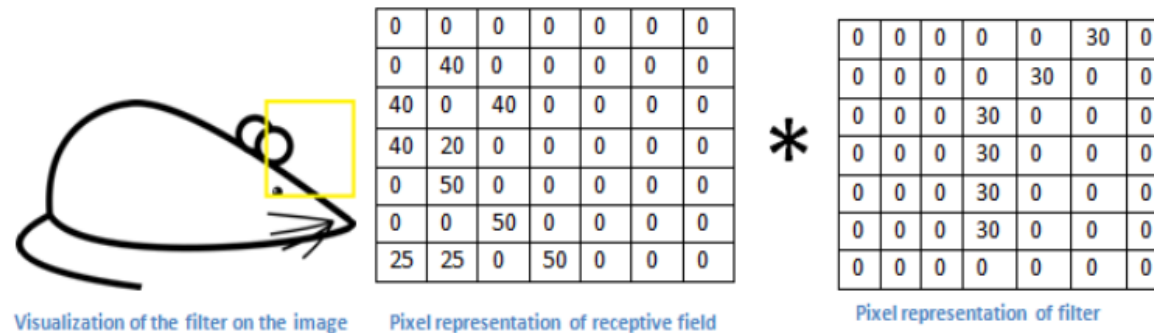
0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Multiplication and Summation = $(50*30)+(50*30)+(50*30)+(20*30)+(50*30) = 6600$ (A large number!)

Convolutional Layer – High Level Perspective

- The value is much lower! This is because there wasn't anything in the image section that responded to the curve detector filter.

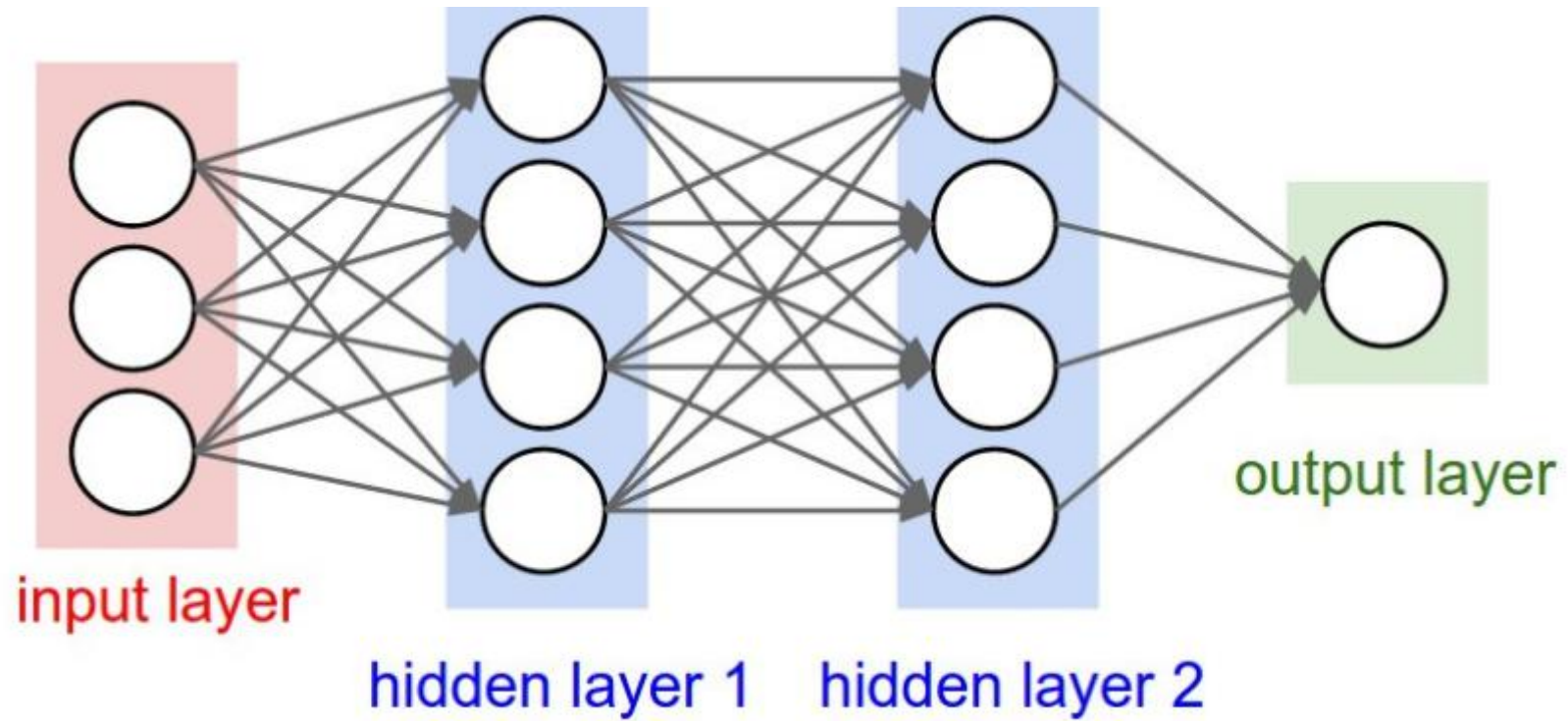


Multiplication and Summation = 0

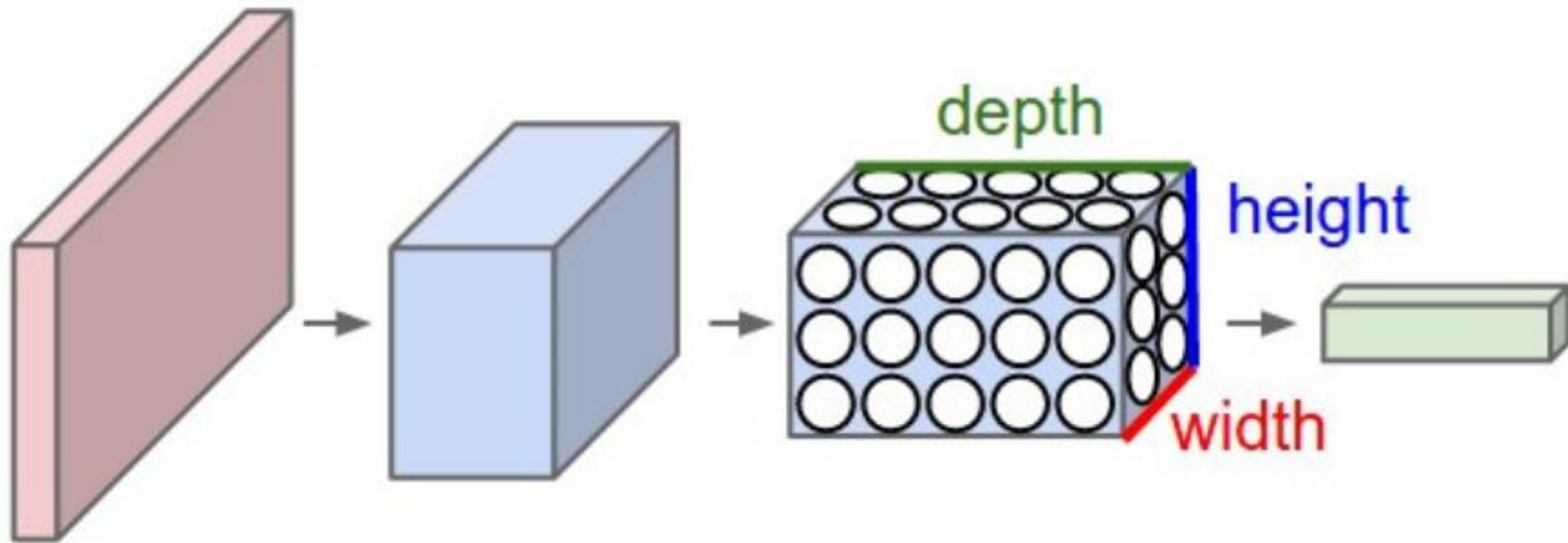
Recognition Task (Images)

- When we look at a picture of a dog, we can classify it as such if the picture has identifiable features such as paws or 4 legs.
- In a similar way, the computer is able to perform image classification by looking for low level features such as edges and curves, and then building up to more abstract concepts through a series of convolutional layers.
- This is a general overview of what a CNN does.

ANN



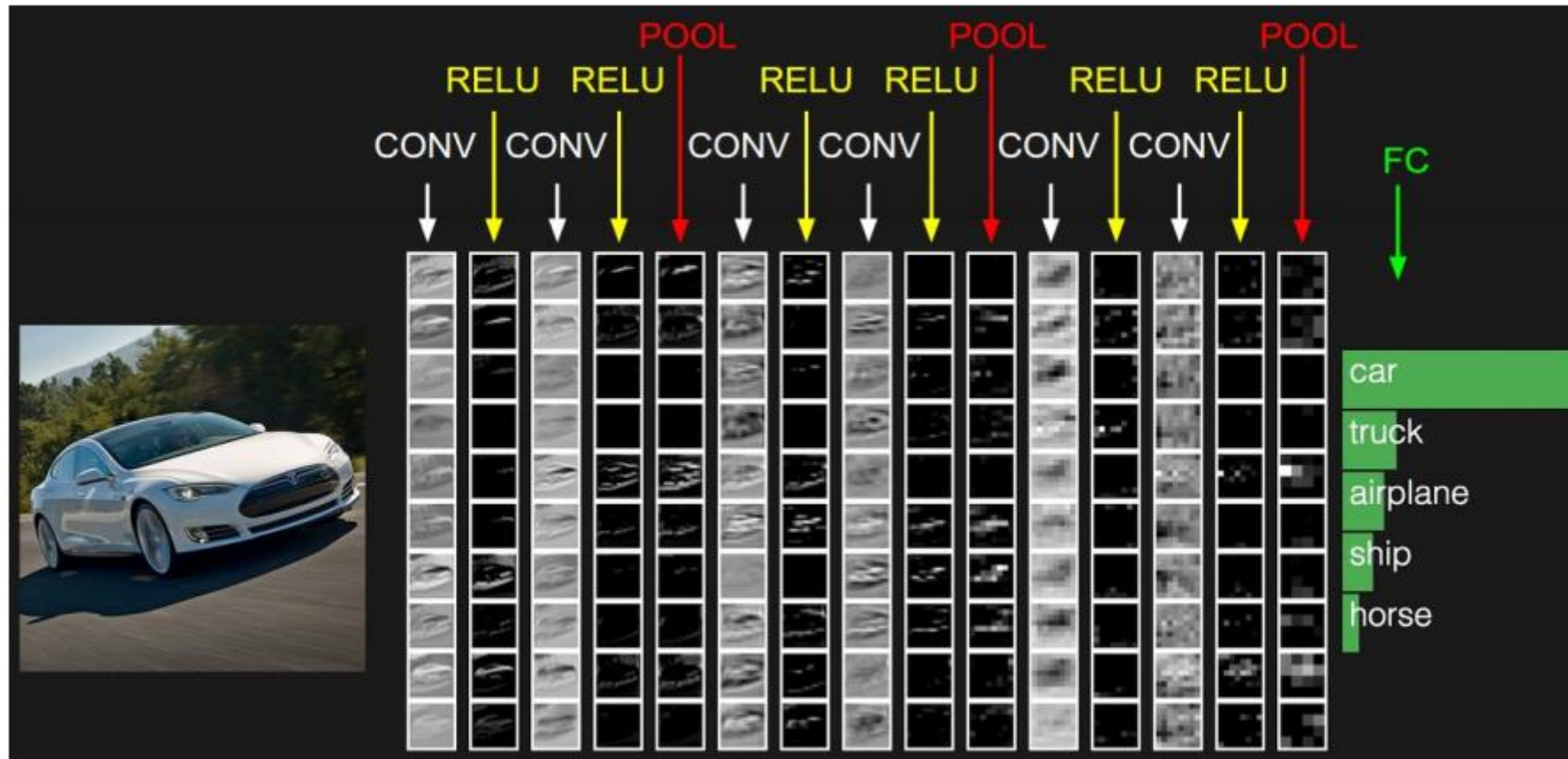
CNN



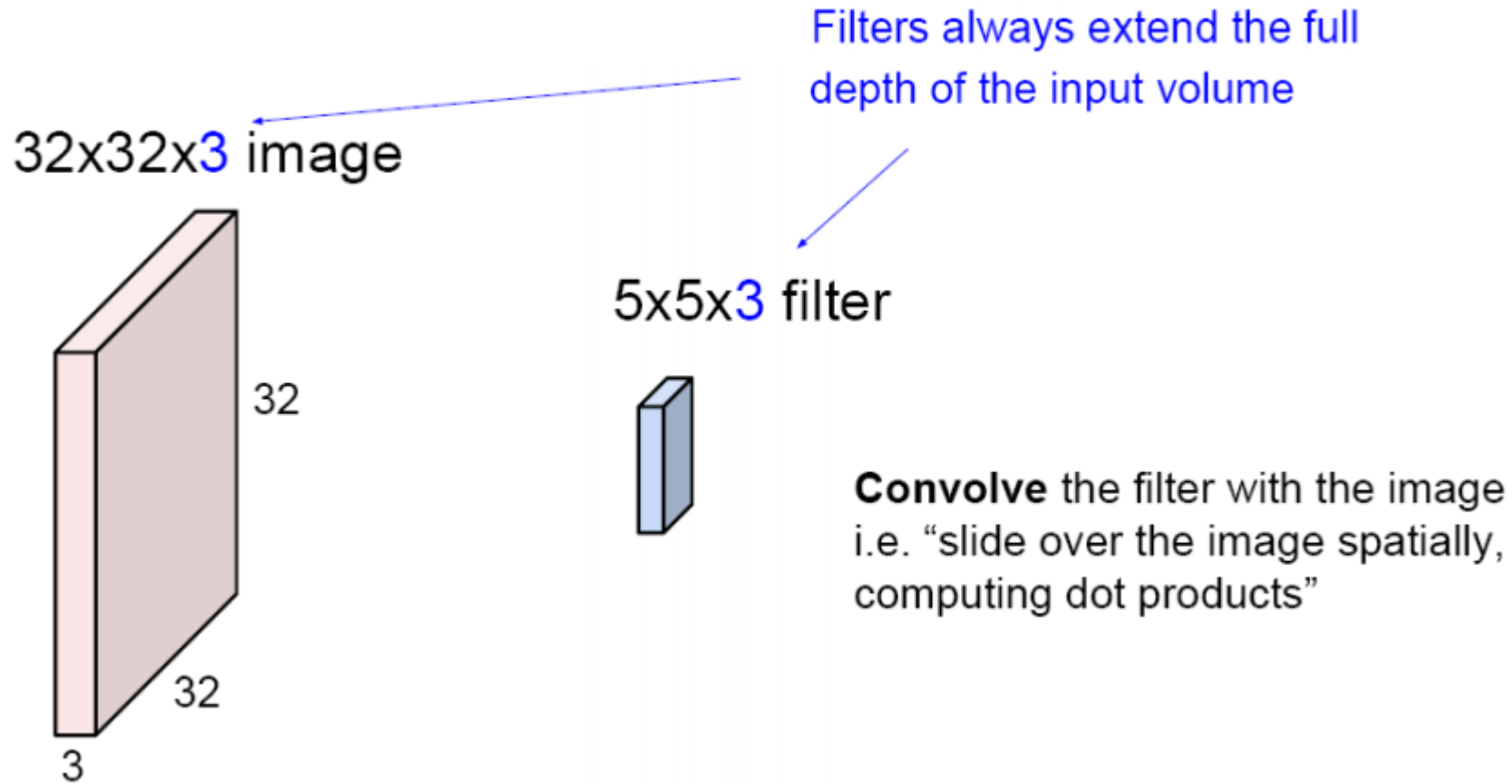
CNN Layers

- Layers
 - Input Layer (**INPUT**)
 - Convolutional Layer (**CONV**)
 - ReLU Layer (**RELU**)
 - Pooling (Downsampling) Layer (**POOL**)
 - Fully connected Layer (**FC**)

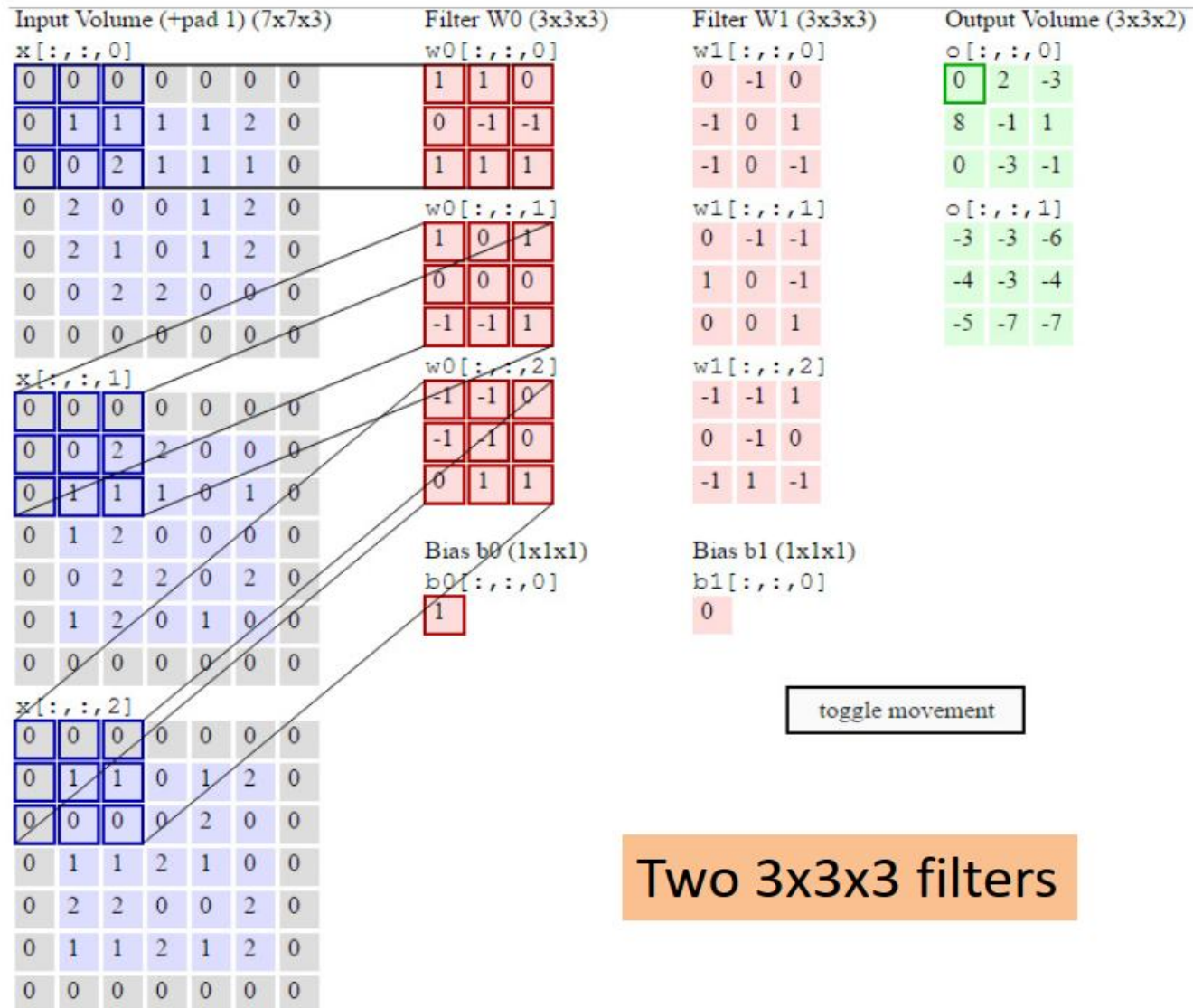
Example ConvNet Architecture



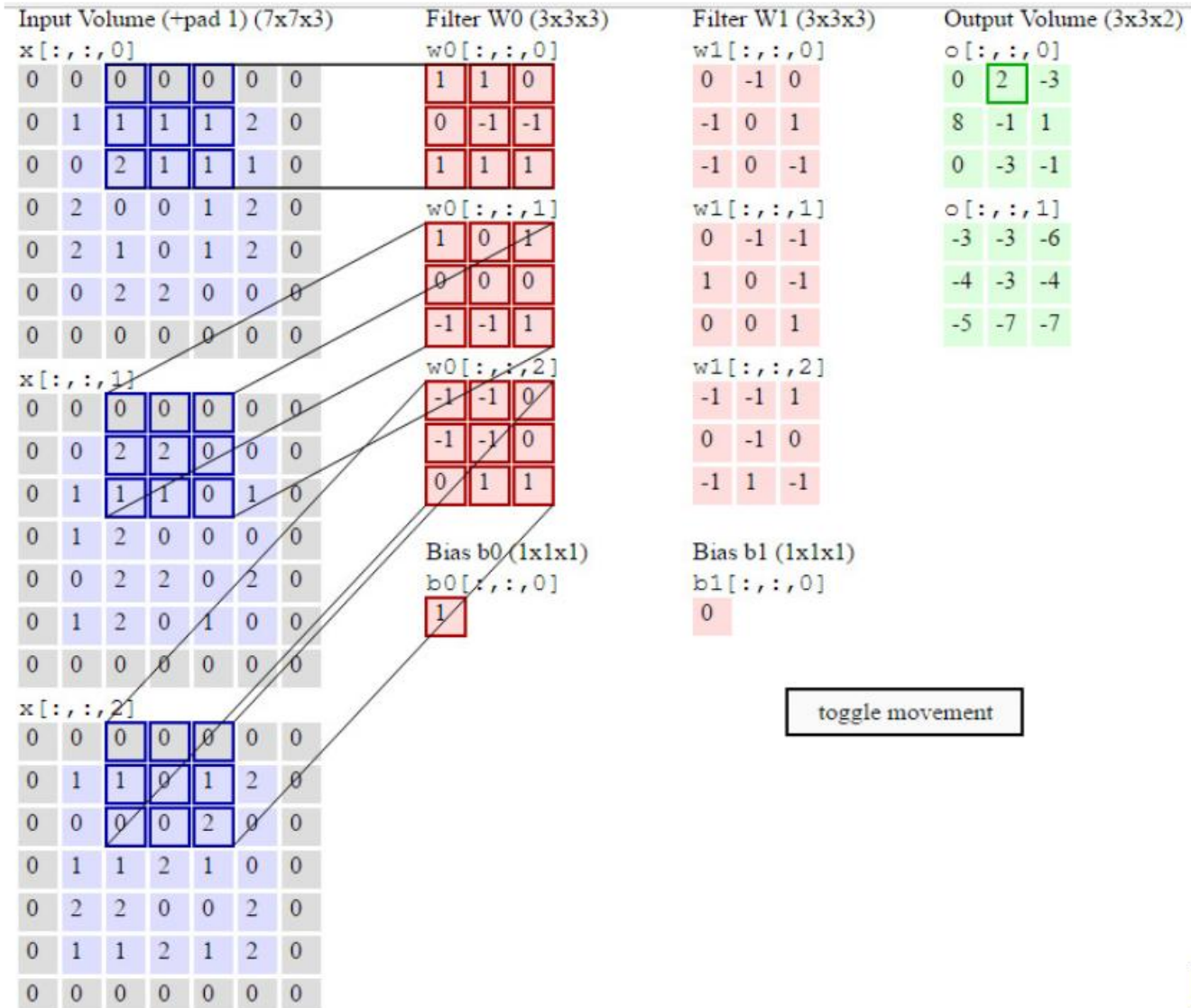
Convolution Layer



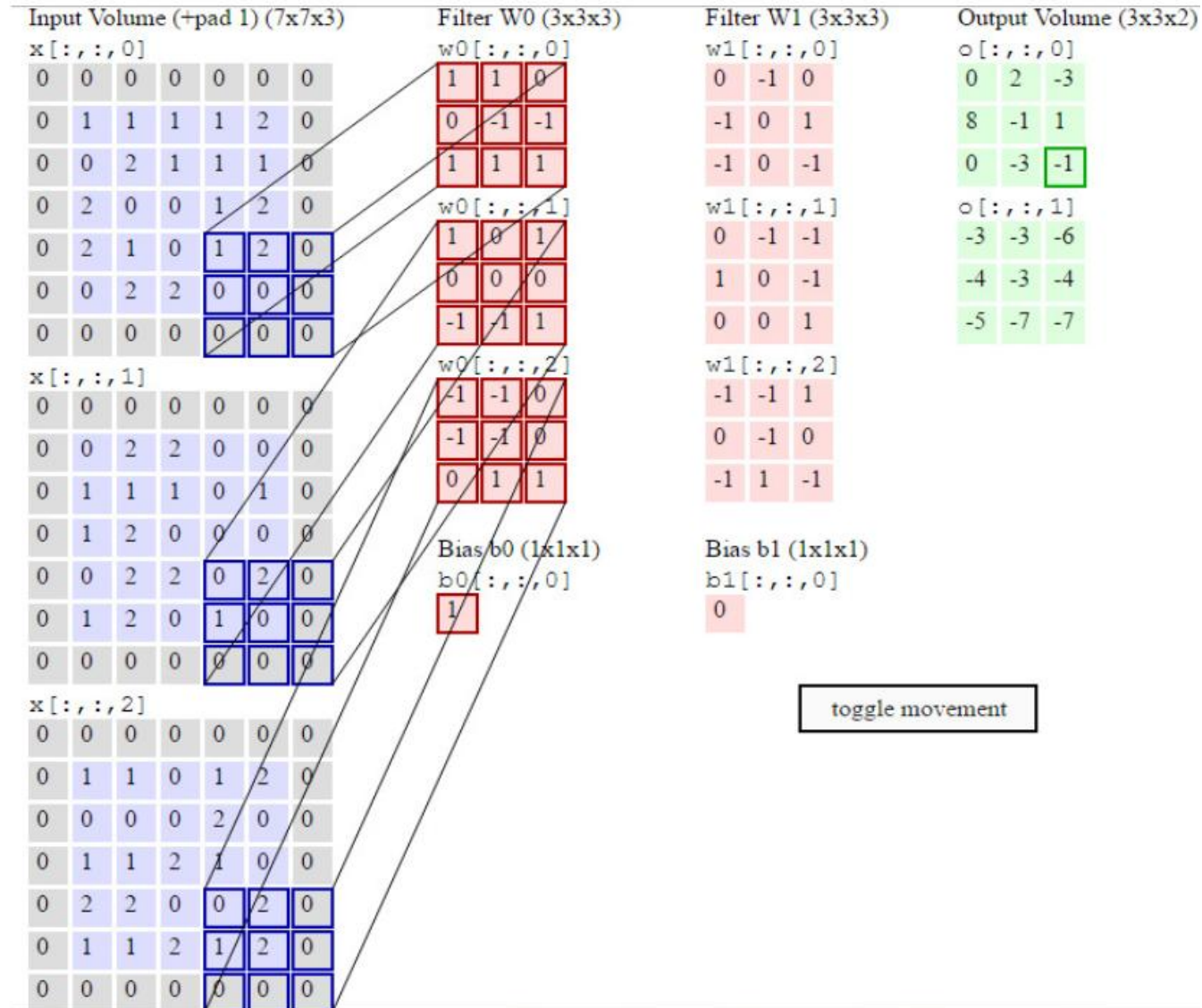
Example



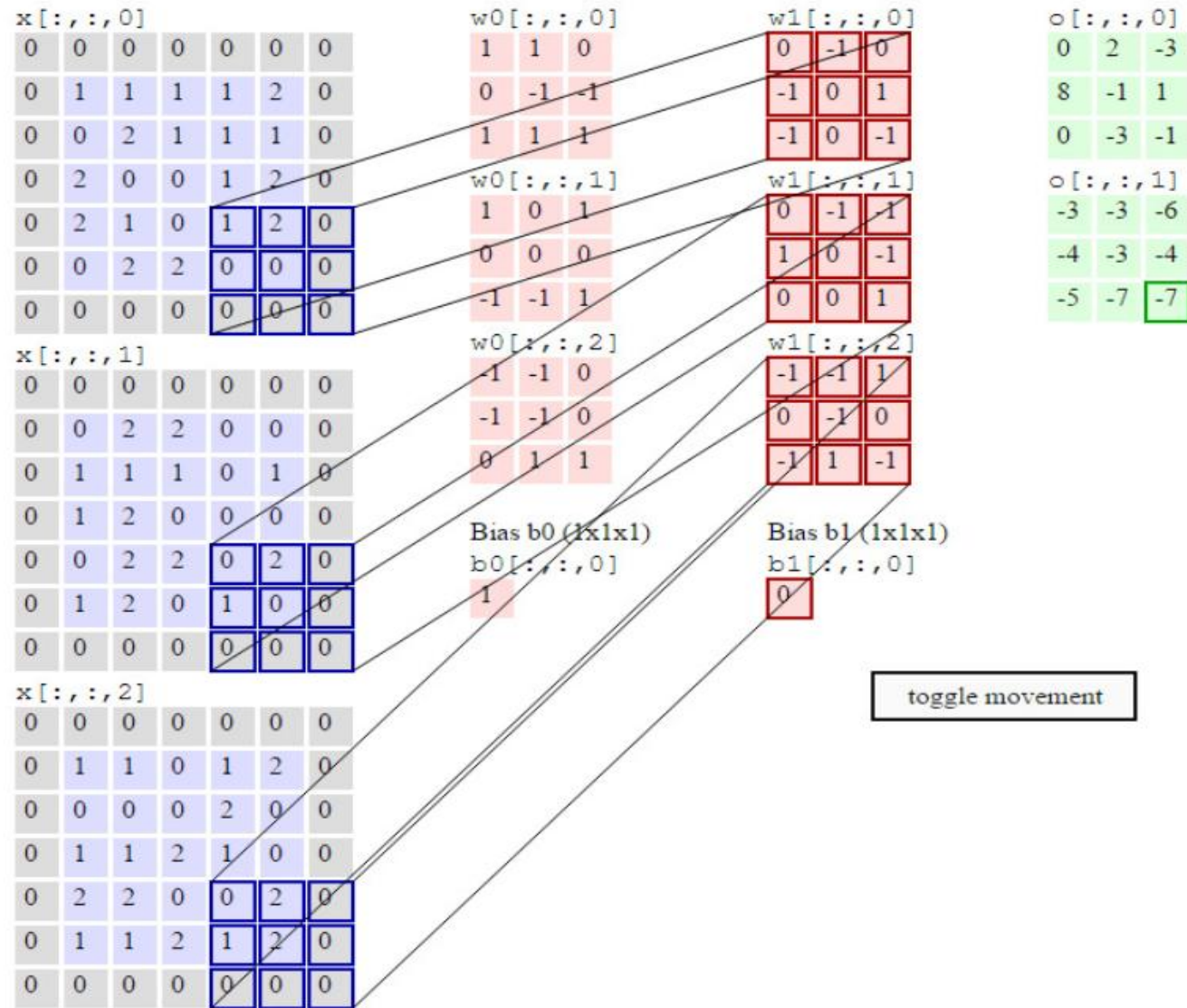
Example



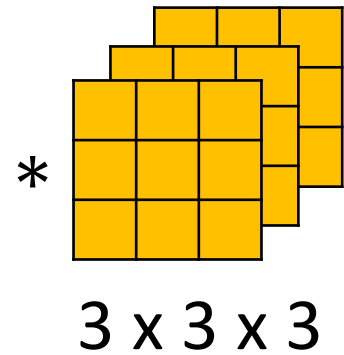
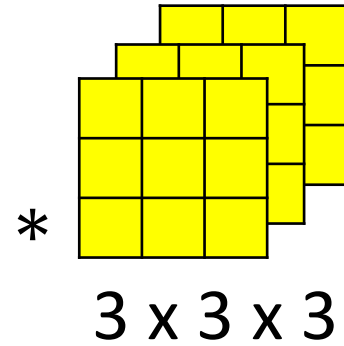
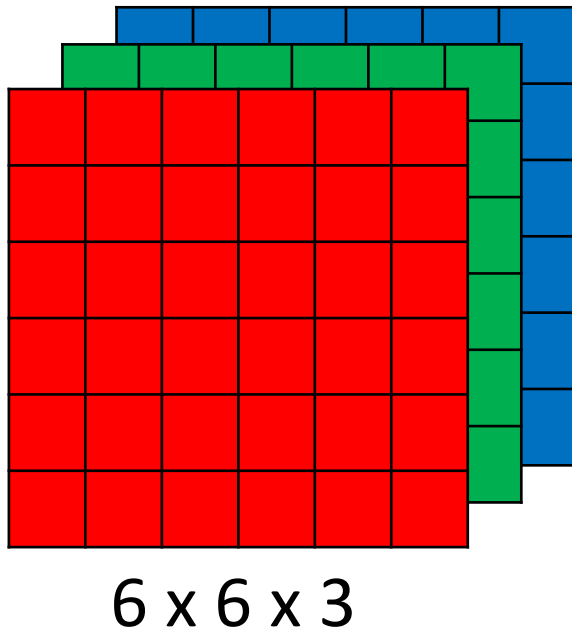
Example



Example

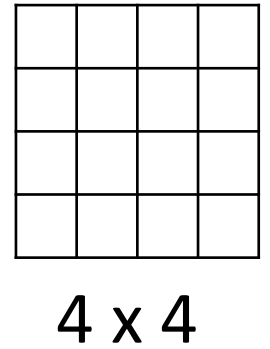


One Conv layer



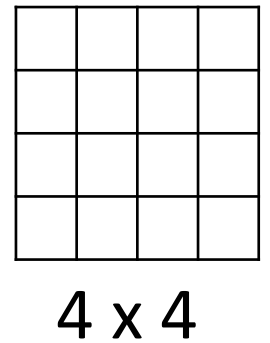
$$\text{ReLU}\left(\begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} + b_1\right)$$

4 x 4



$$\text{ReLU}\left(\begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} + b_2\right)$$

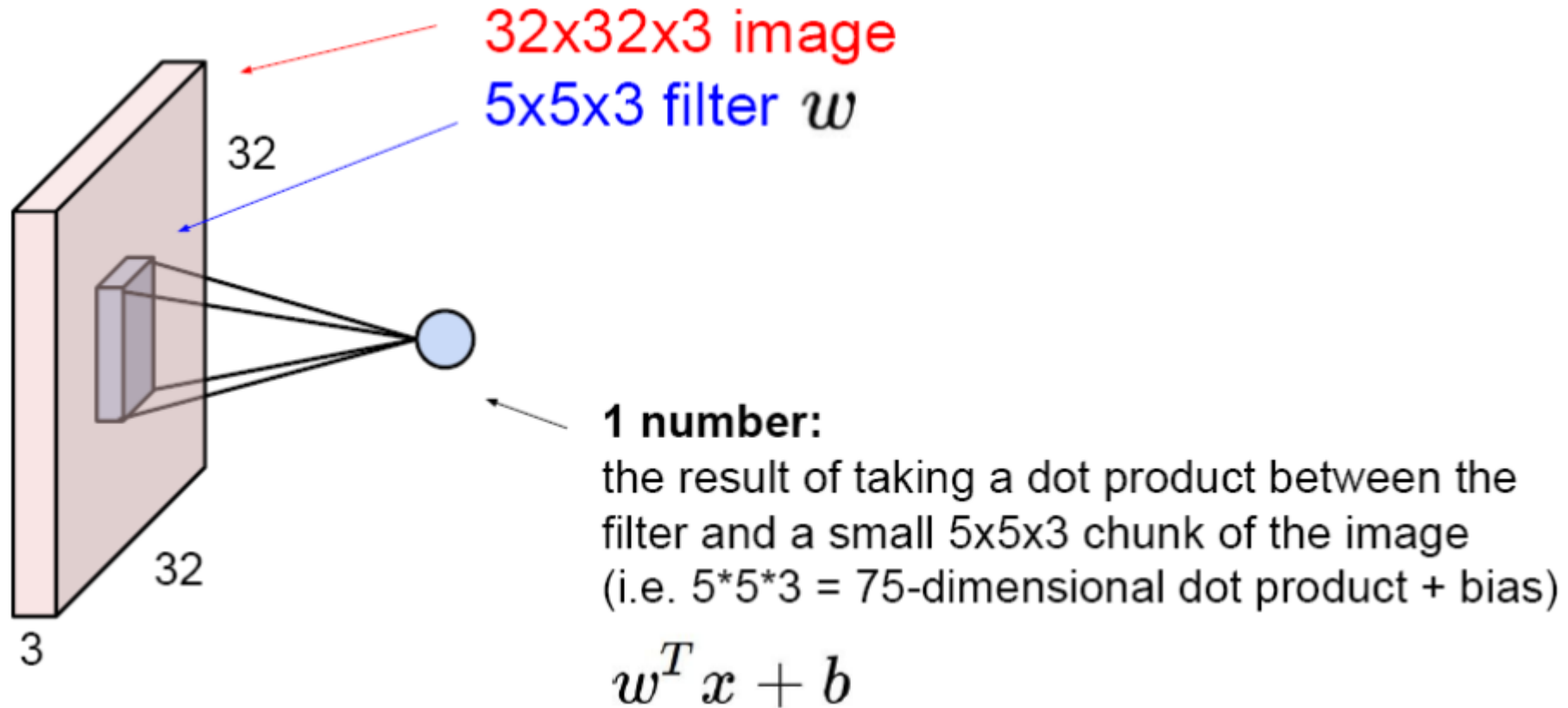
4 x 4



Convolution Layer

- Input is a 32 x 32 x 3 array of pixel values
- Imagine a flashlight that is shining over the top left of the image.
- Let's say that this flashlight shines covers a 5 x 5 area and is sliding across all the areas of the input image
- In machine learning terms, this flashlight is called a **filter** (or sometimes referred to as a **kernel**) and the region that it is shining over is called the **receptive field**.

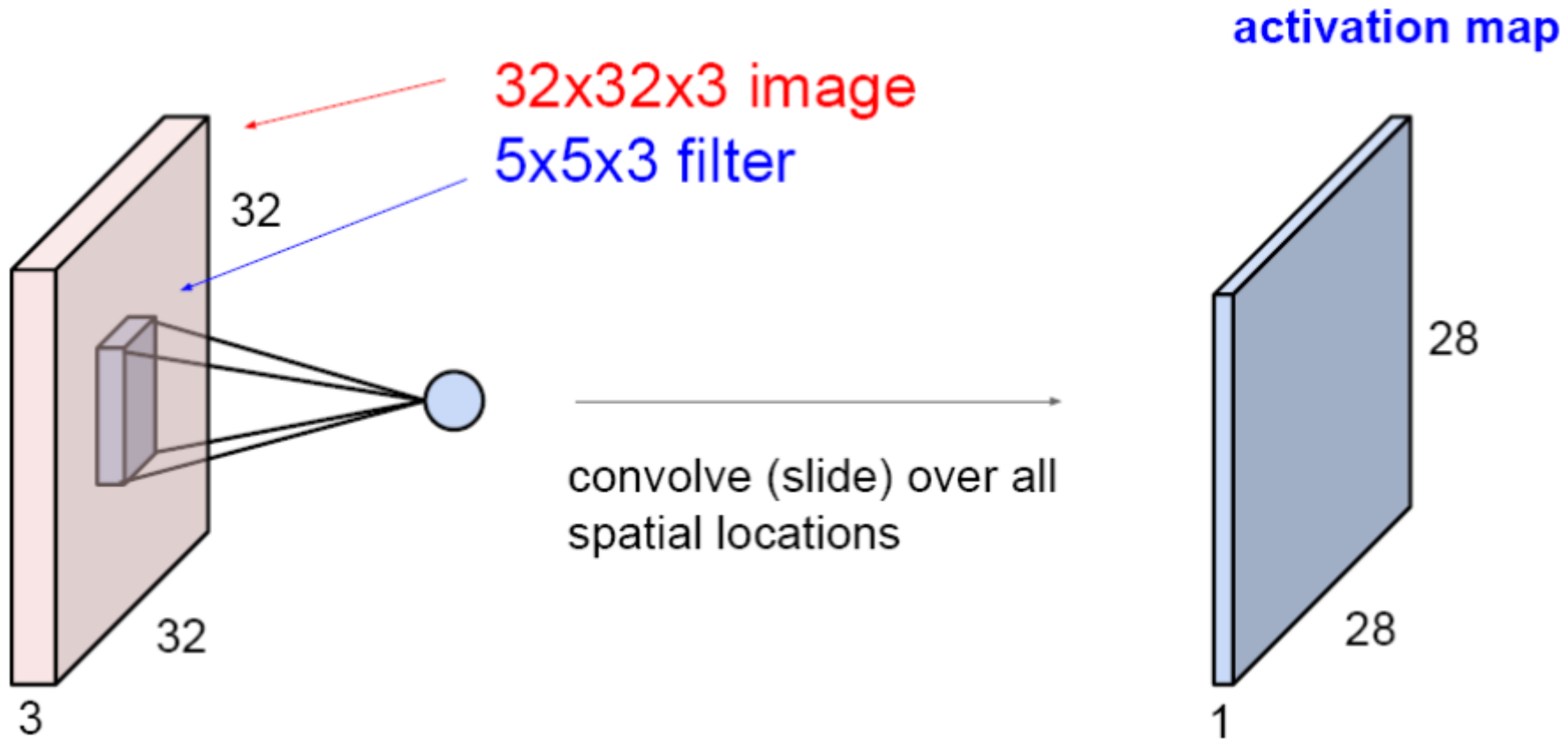
Convolution Layer



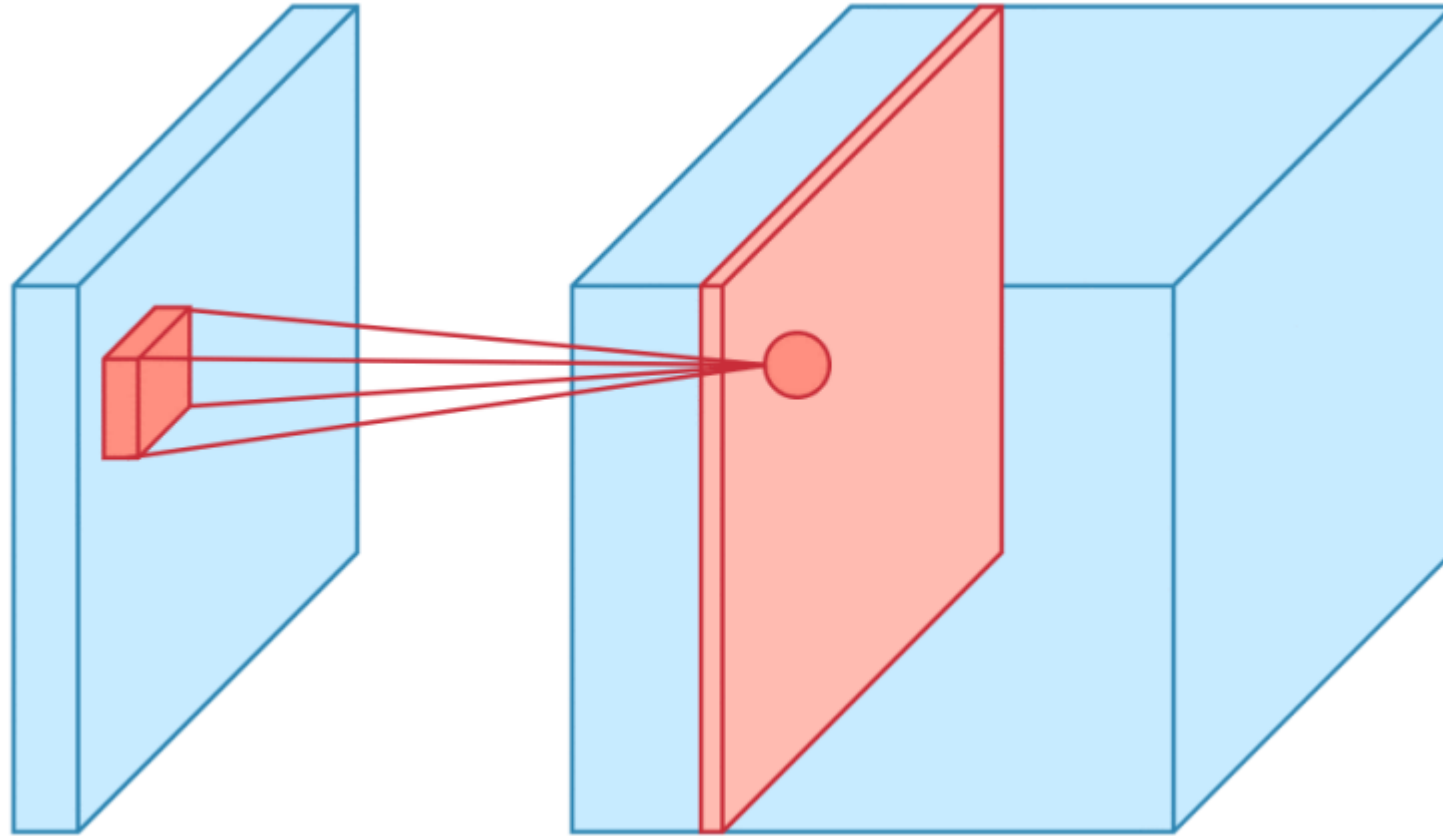
Convolution Layer

- After sliding the filter over all the locations, we're left with a $28 \times 28 \times 1$ array of numbers, which we call an **activation map** or **feature map**.
- The reason we get a 28×28 array is that there are 784 different locations that a 5×5 filter can fit on a 32×32 input image.
- These 784 numbers are mapped to a 28×28 array.

Convolution Layer

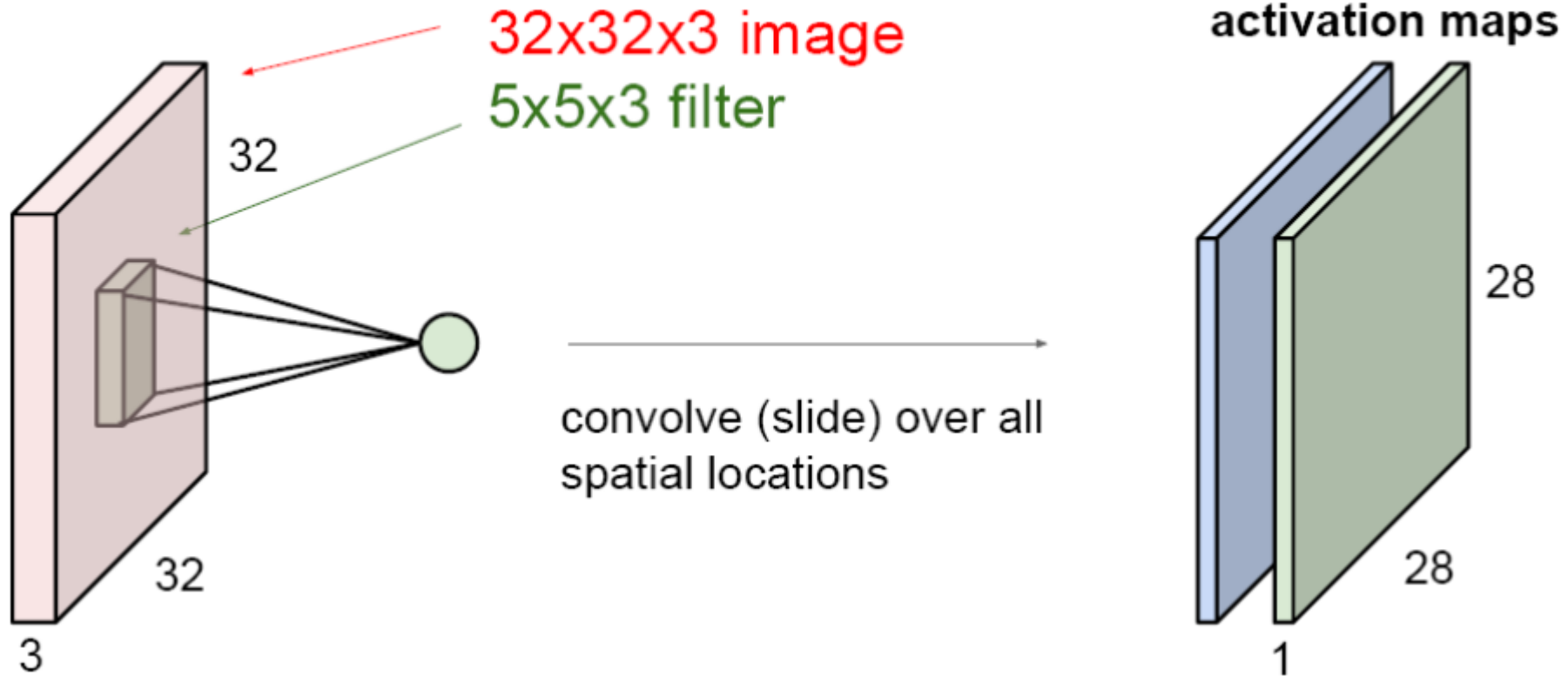


Convolution Layer



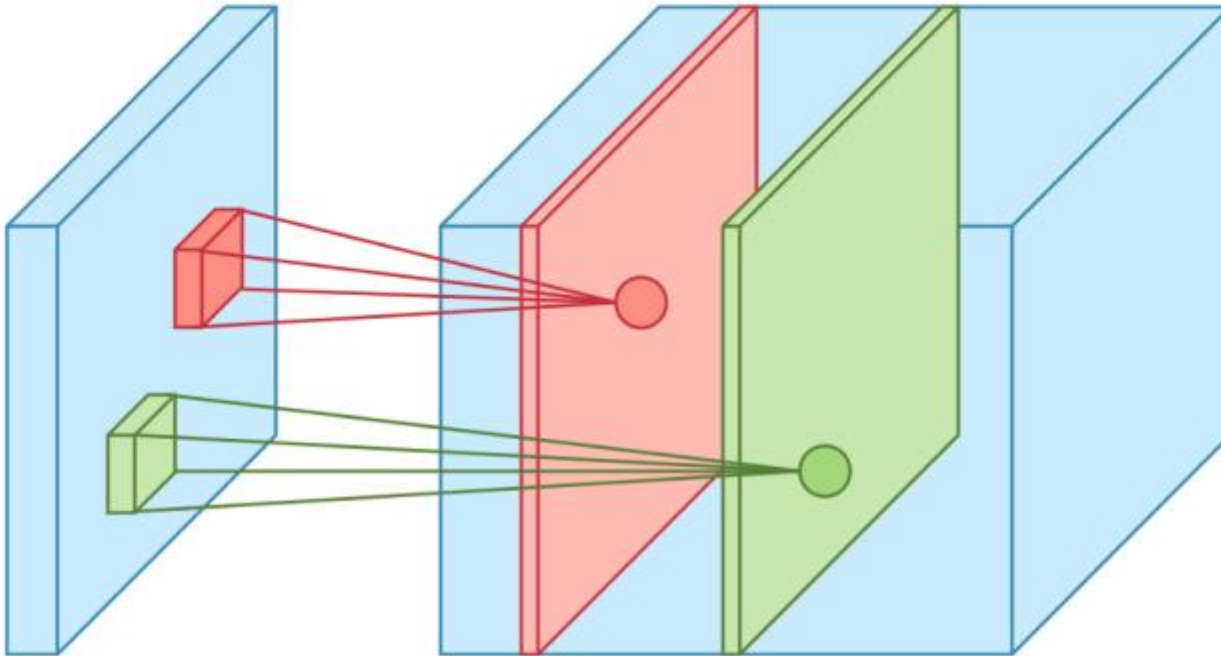
Convolution Layer

Consider a second filter



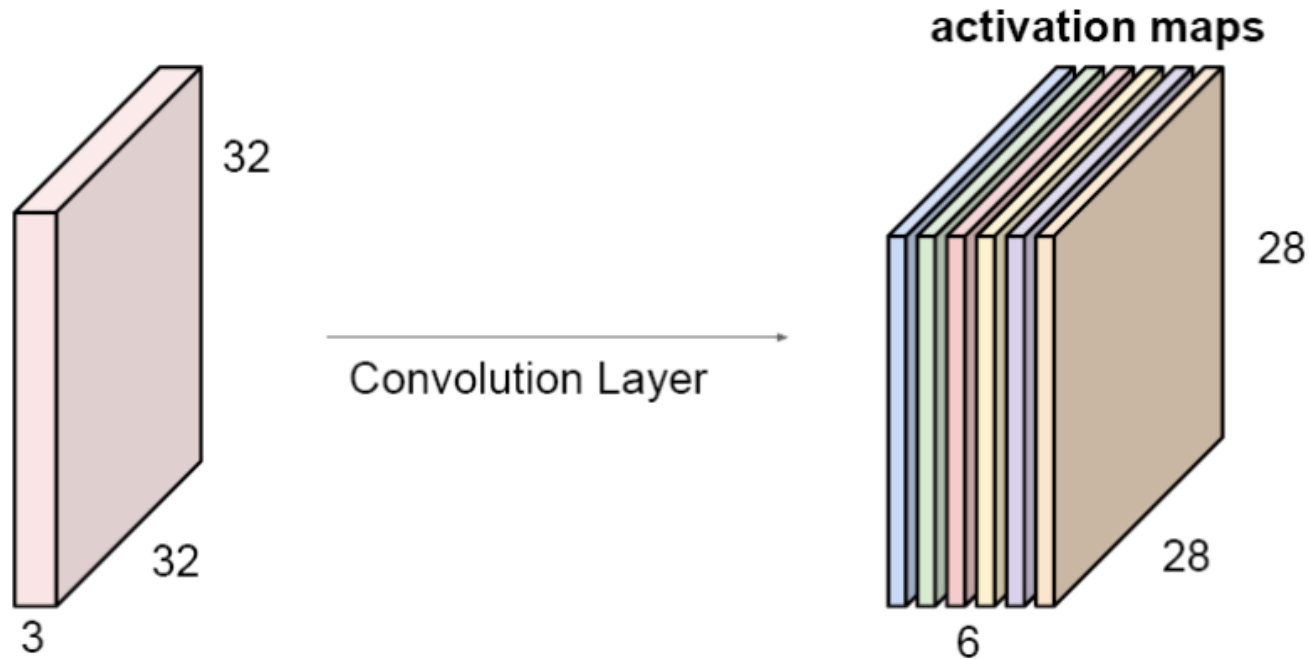
Convolution Layer

- We perform multiple convolutions on an input, each using a different filter and resulting in a distinct feature map. We then stack all these feature maps together and that becomes the final output of the convolution layer.



Convolution Layer

- If we had 6 5x5 filters, we'll get 6 separate activation maps:

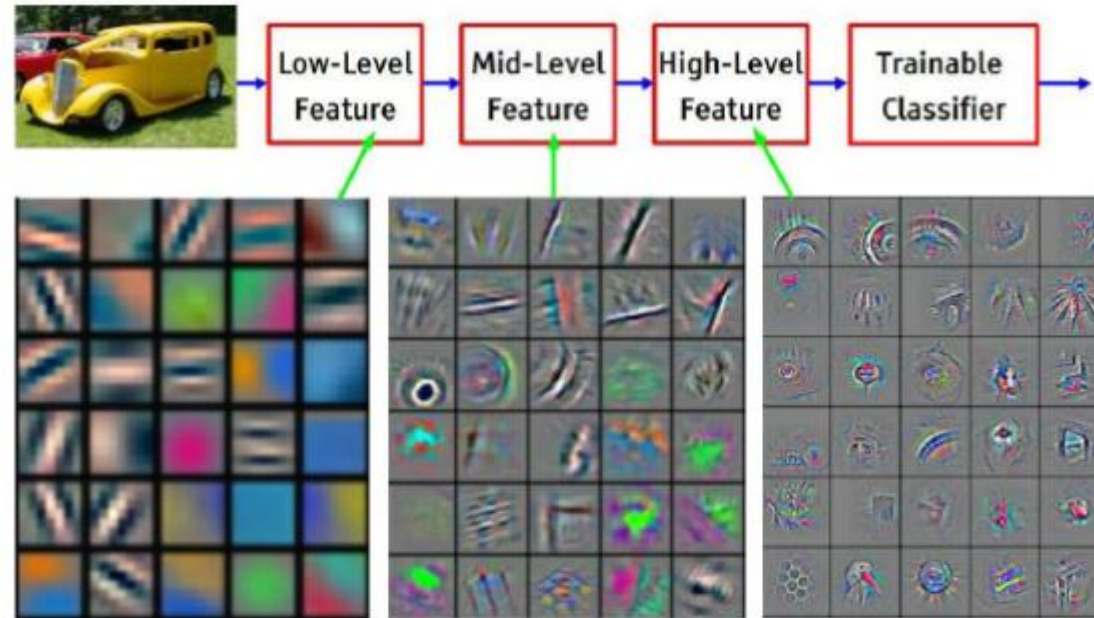


We stack these up to get a "new image" of size 28x28x6!

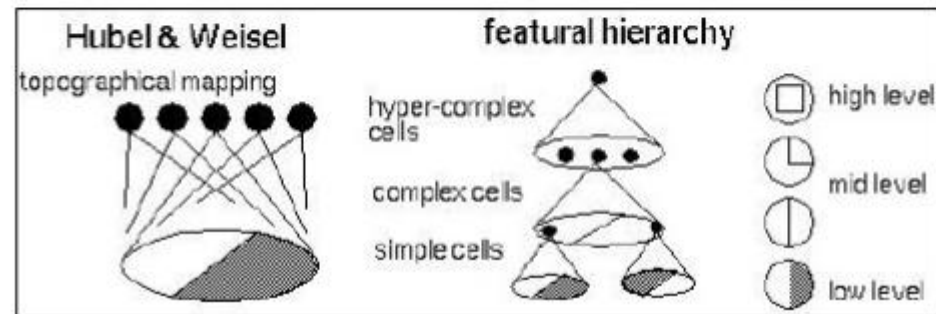
Convolution Layer– Feature Detector

- Each of these filters can be thought of as **feature identifiers**
- Features: Things like straight edges, simple colors, and curves.
- Let's say our first filter is 7 x 7 x 3 and is going to be a curve detector.
- As a curve detector, the filter will have a pixel structure in which there will be higher numerical values along the area that is a shape of a curve

Convolution Layer– Feature Detector



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

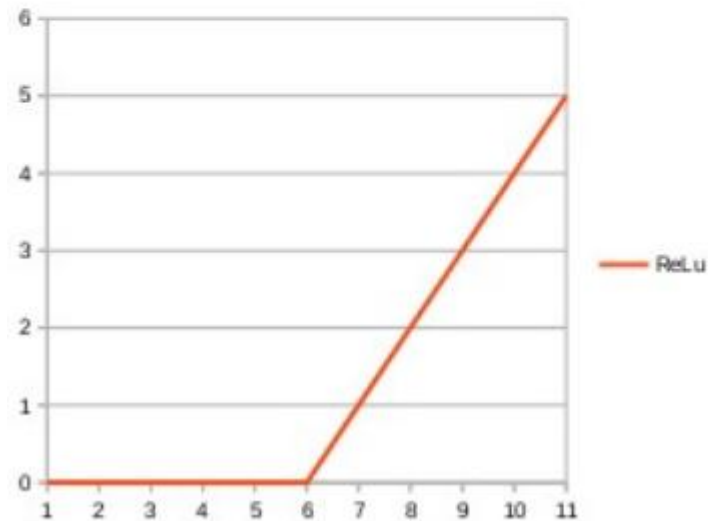


ReLU (Rectified Linear Units) Layers

- Functions like tanh and sigmoid have been widely used in ANNs.
- Researchers found out that ReLU layers work far better because the network is able to train a lot faster (because of the computational efficiency) without making a significant difference to the accuracy.

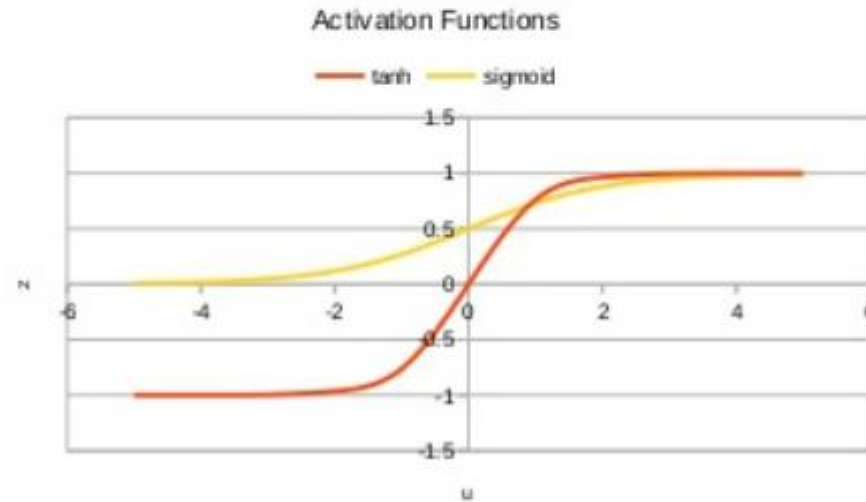
Activation Function Examples

Relu



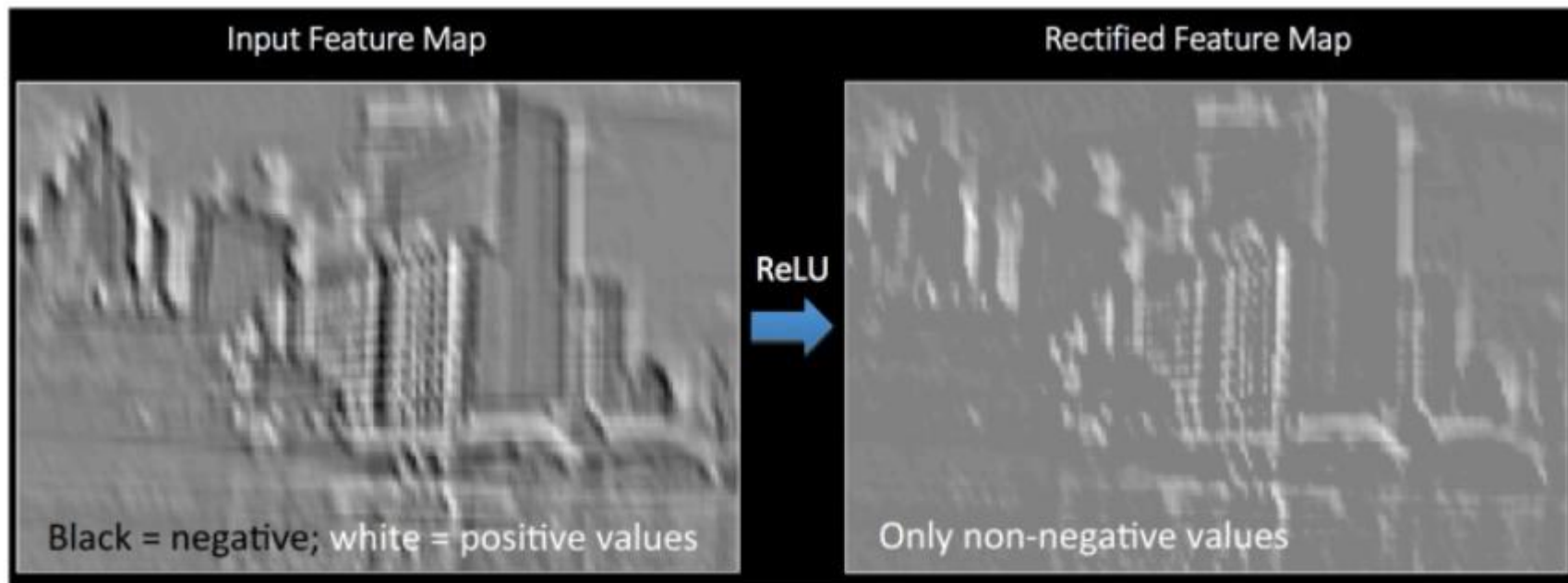
tanh

sigmoid



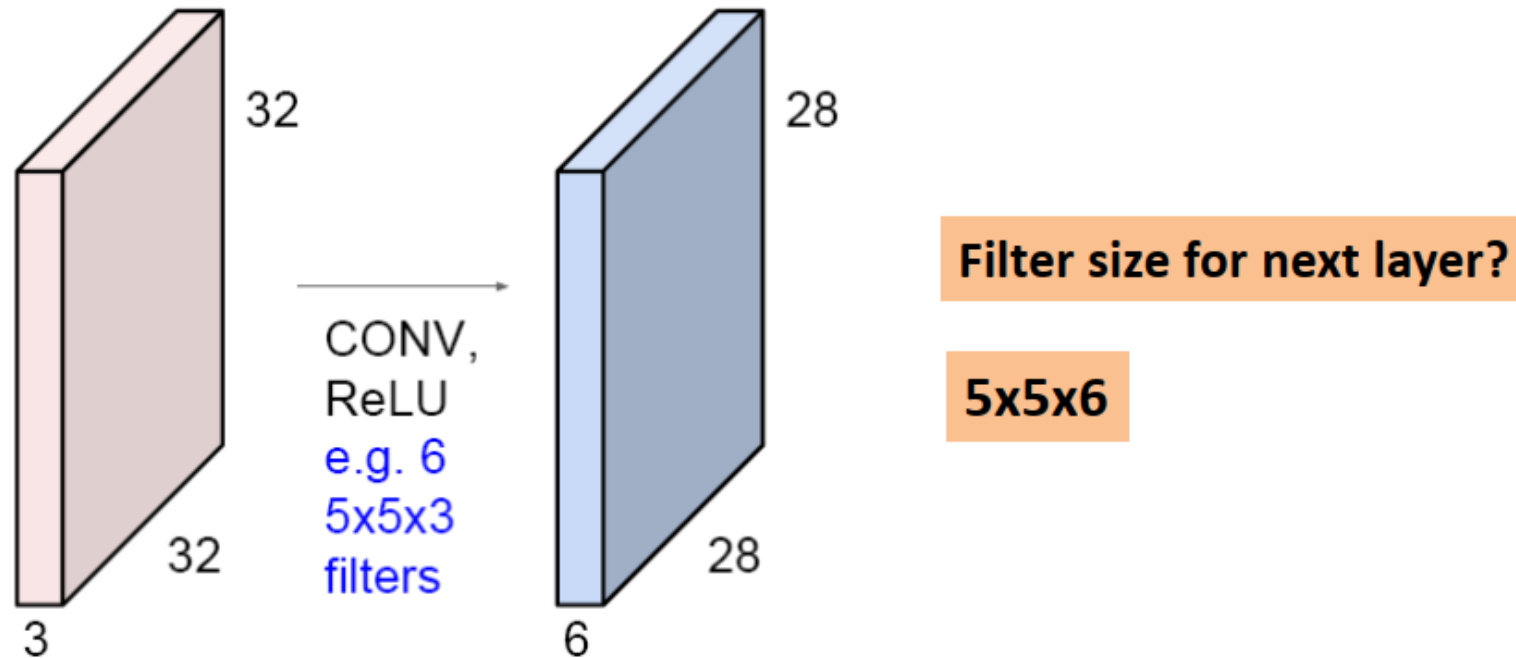
ReLU (Rectified Linear Units) Layers

- The ReLU layer applies the function $f(x) = \max(0, x)$ to all of the values in the input volume.
- In basic terms, this layer just changes all the negative activations to 0.



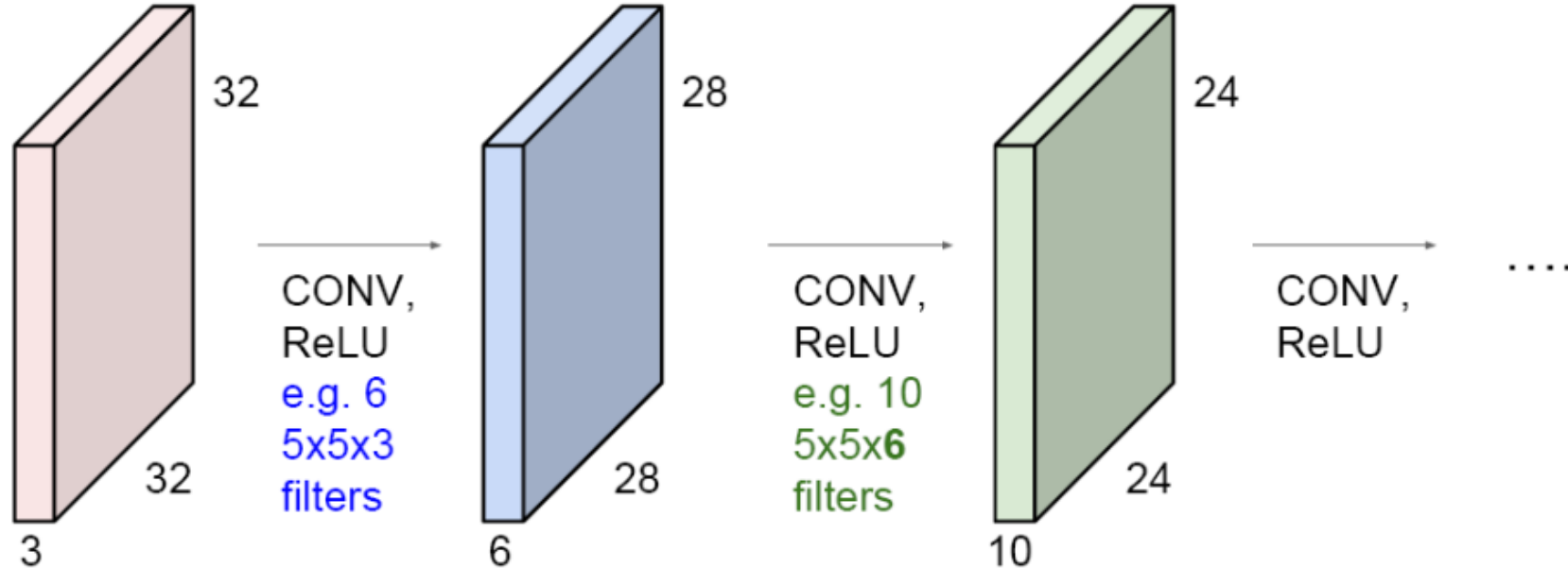
Conv + ReLU

- ConvNet is a sequence of Convolution Layers, interspersed with activation functions

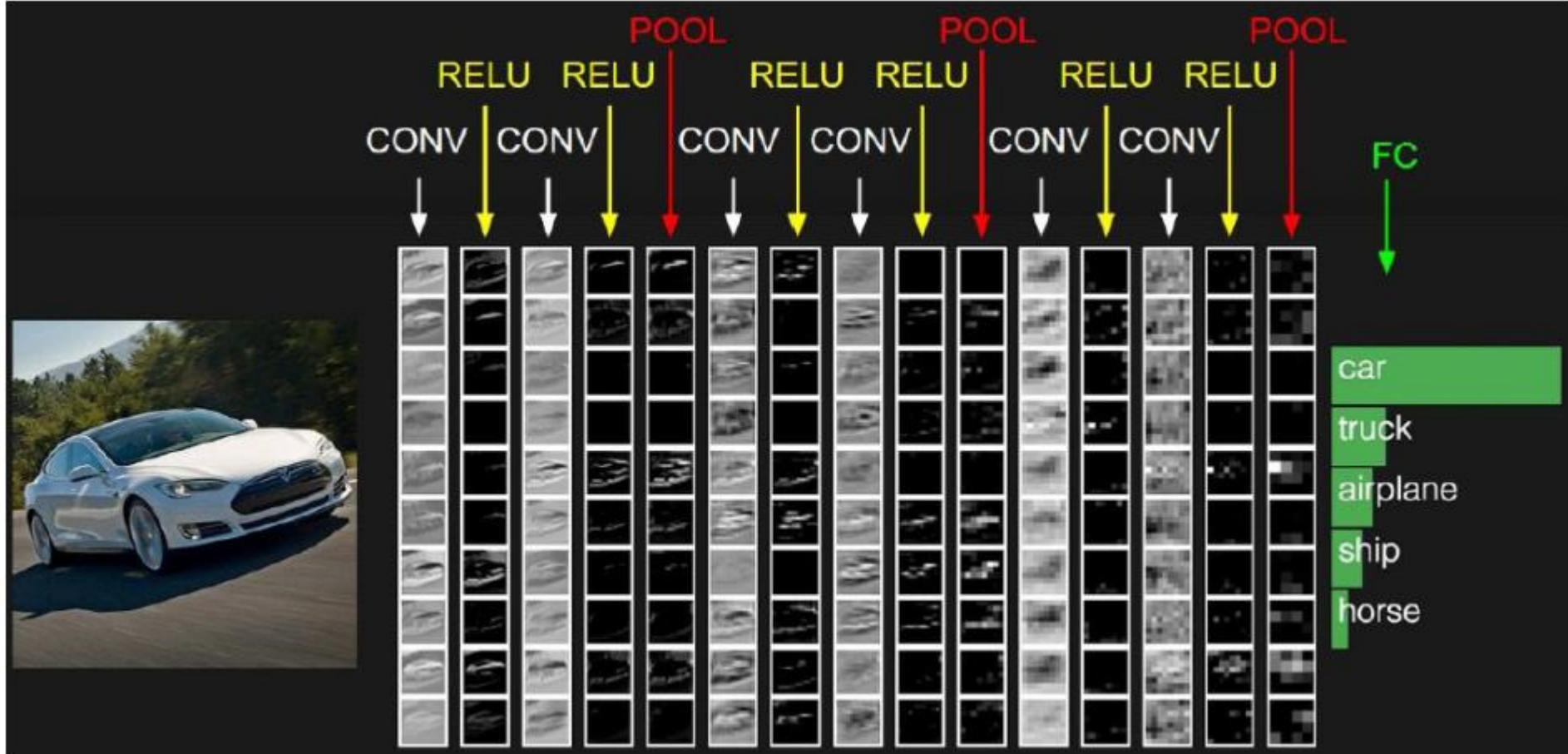


Conv + ReLU

- ConvNet is a sequence of Convolution Layers, interspersed with activation functions



Conv + ReLU

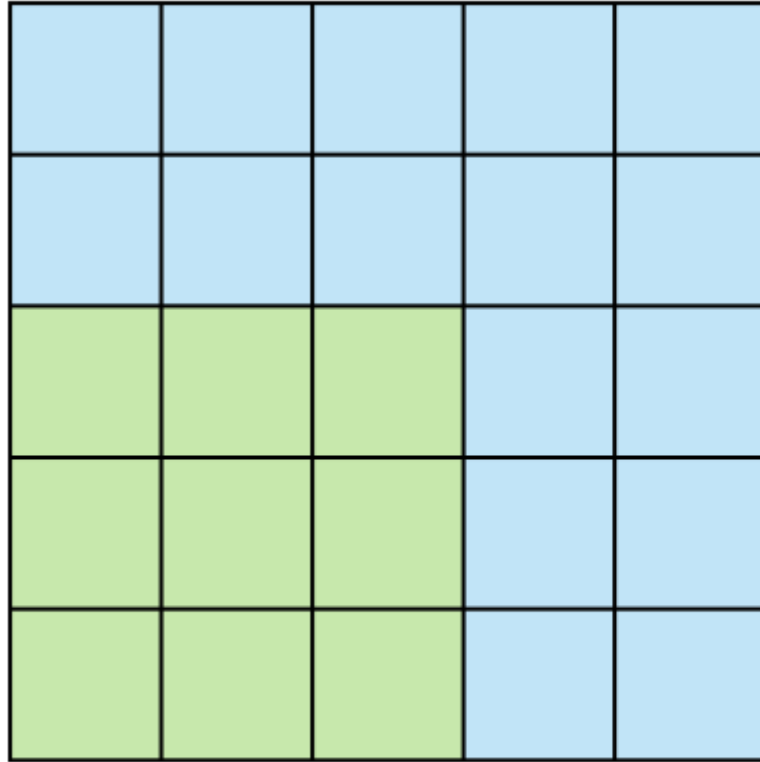


Spatial Dimensions – Closer Look

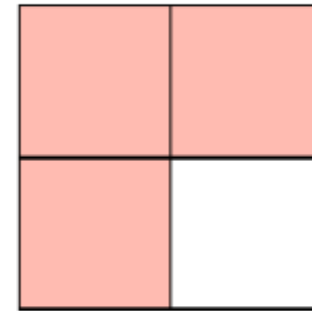
Three Parameters

- Depth
- Stride
- Padding

Stride

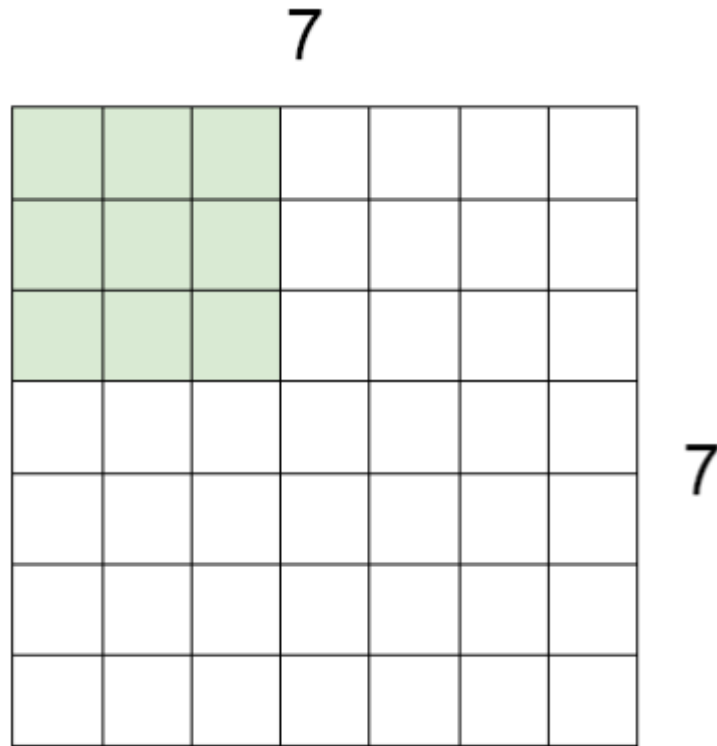


Stride 2



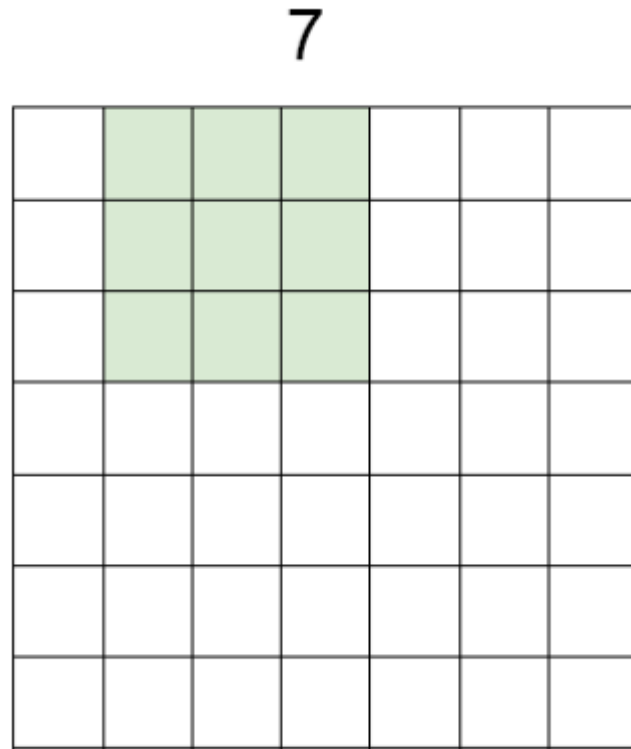
Feature Map

Spatial Dimensions – Closer Look



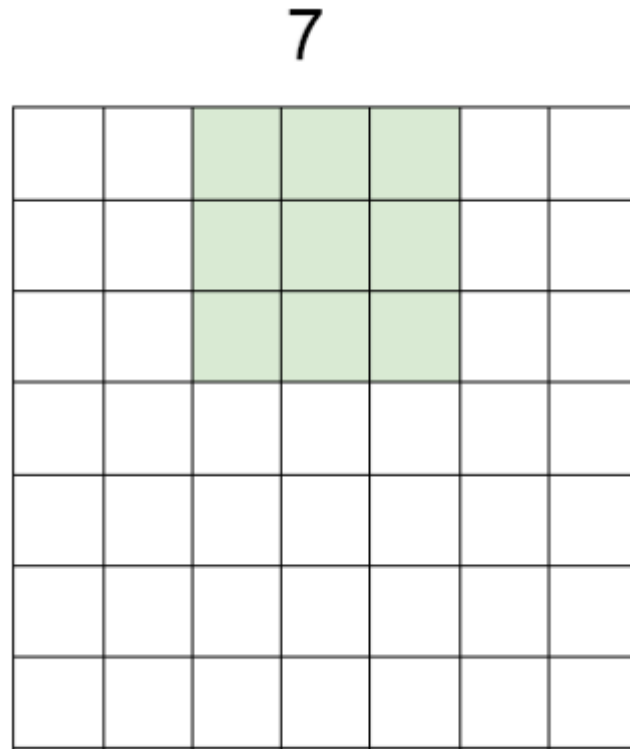
7x7 input (spatially)
assume 3x3 filter

Spatial Dimensions – Closer Look



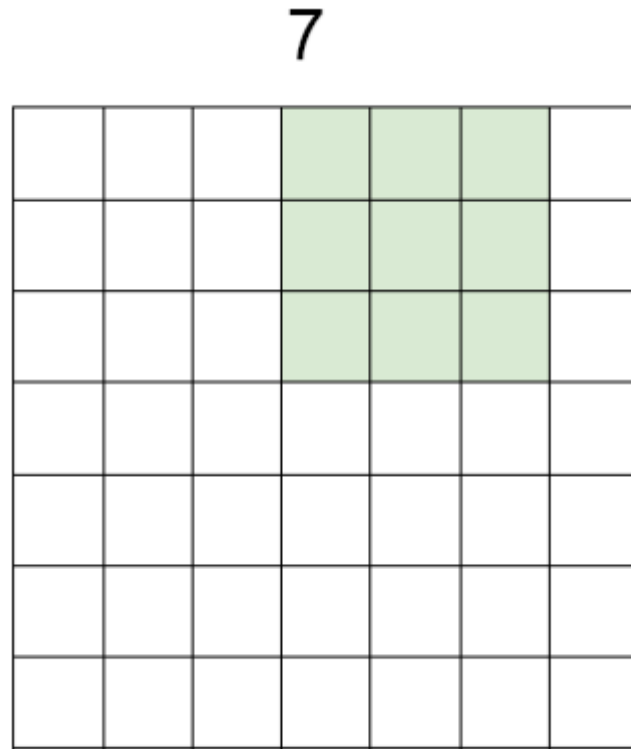
7x7 input (spatially)
assume 3x3 filter

Spatial Dimensions – Closer Look



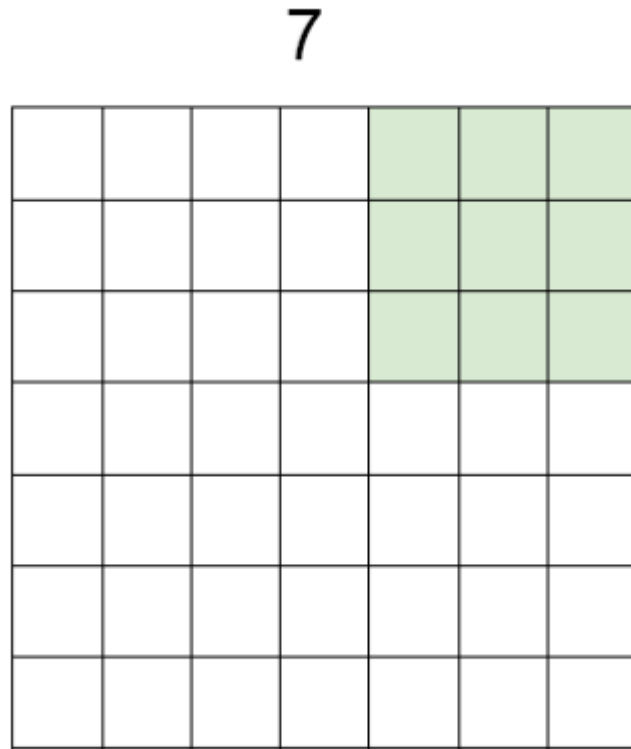
7x7 input (spatially)
assume 3x3 filter

Spatial Dimensions – Closer Look



7x7 input (spatially)
assume 3x3 filter

Spatial Dimensions – Closer Look



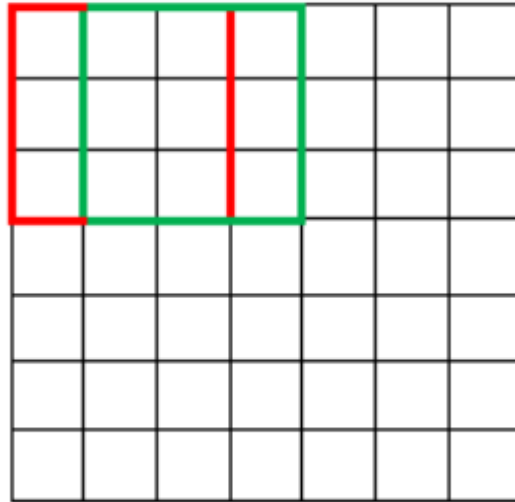
7x7 input (spatially)
assume 3x3 filter

=> 5x5 output

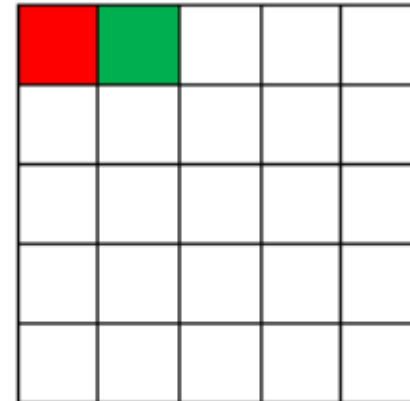
Spatial Dimensions – Closer Look

- 3x3 Filter

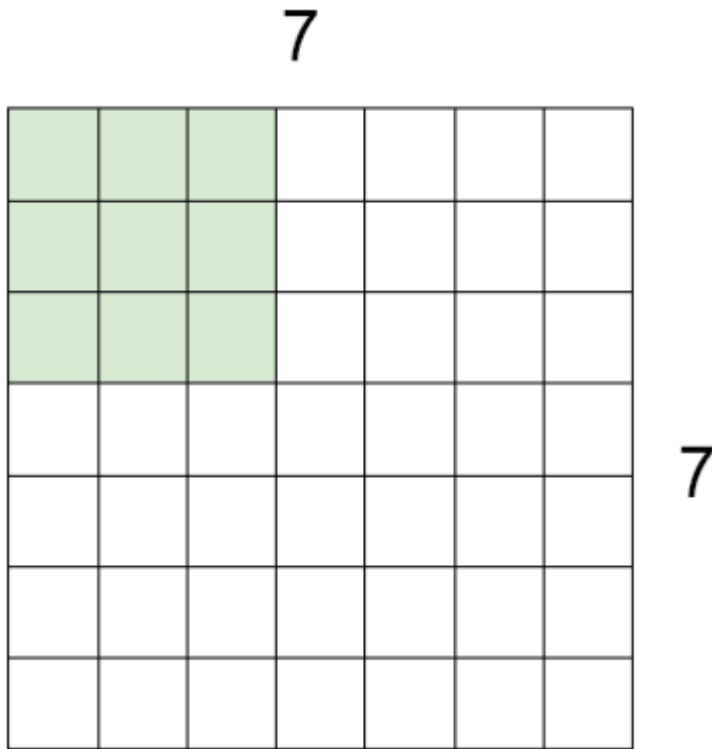
7 x 7 Input Volume



5 x 5 Output Volume

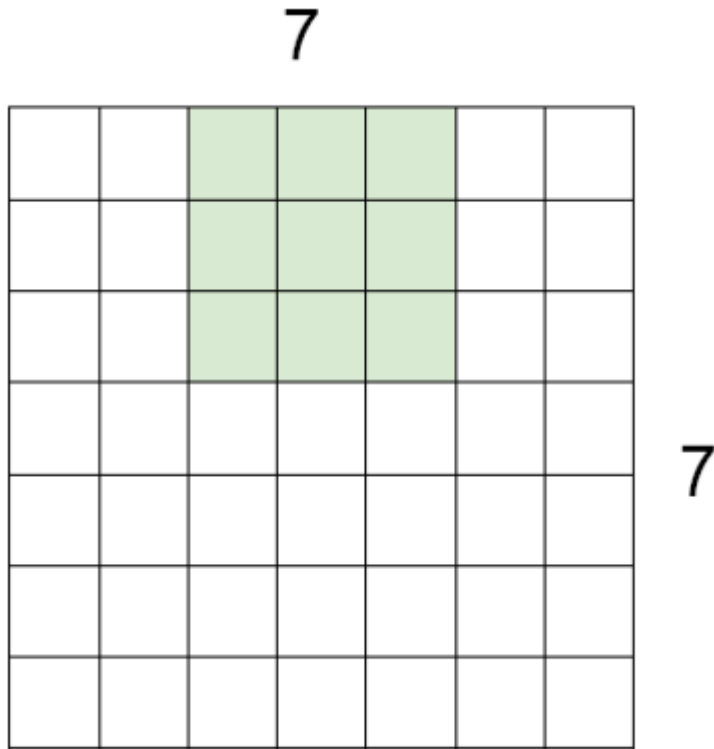


Spatial Dimensions – Closer Look



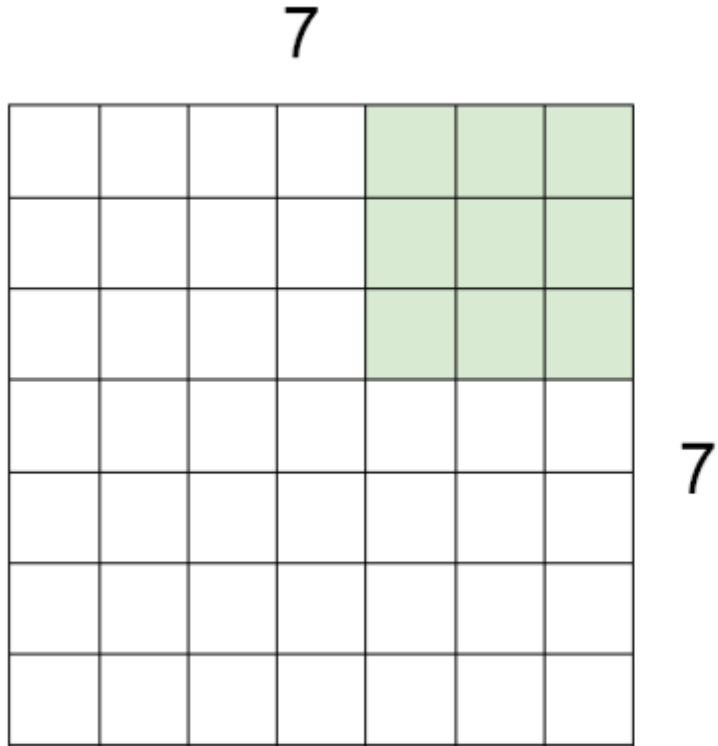
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Spatial Dimensions – Closer Look



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Spatial Dimensions – Closer Look

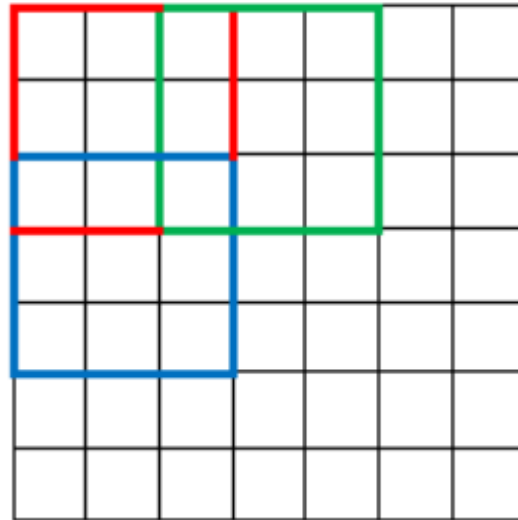


7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

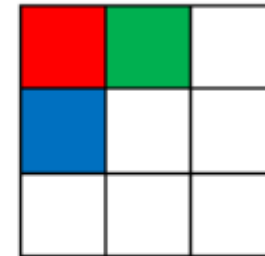
Spatial Dimensions – Closer Look

- 3x3 Filter – Stride increased to 2

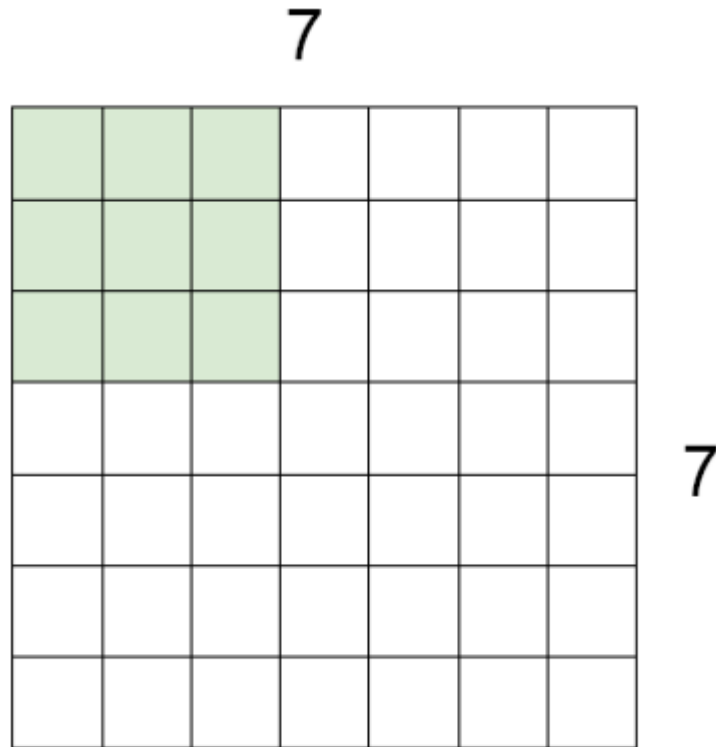
7 x 7 Input Volume



3 x 3 Output Volume

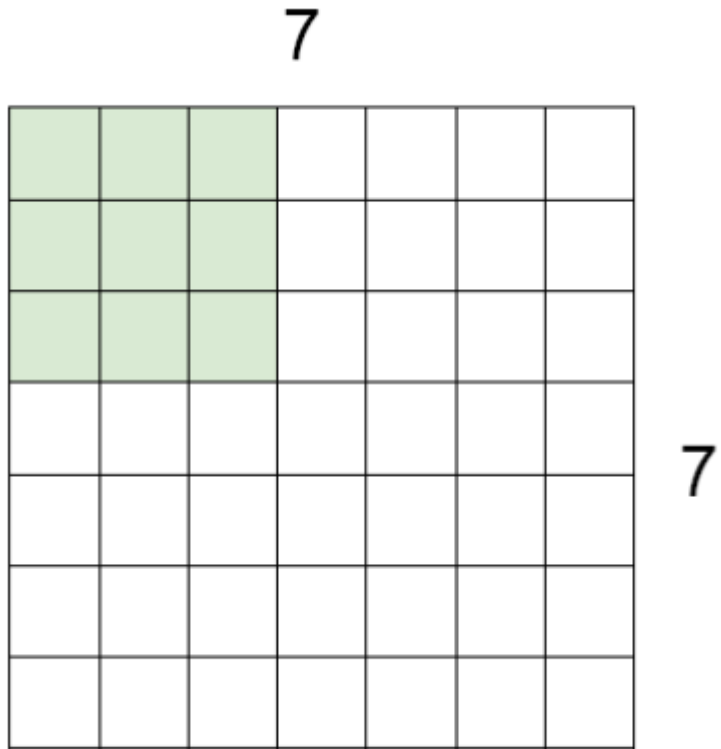


Spatial Dimensions – Closer Look



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

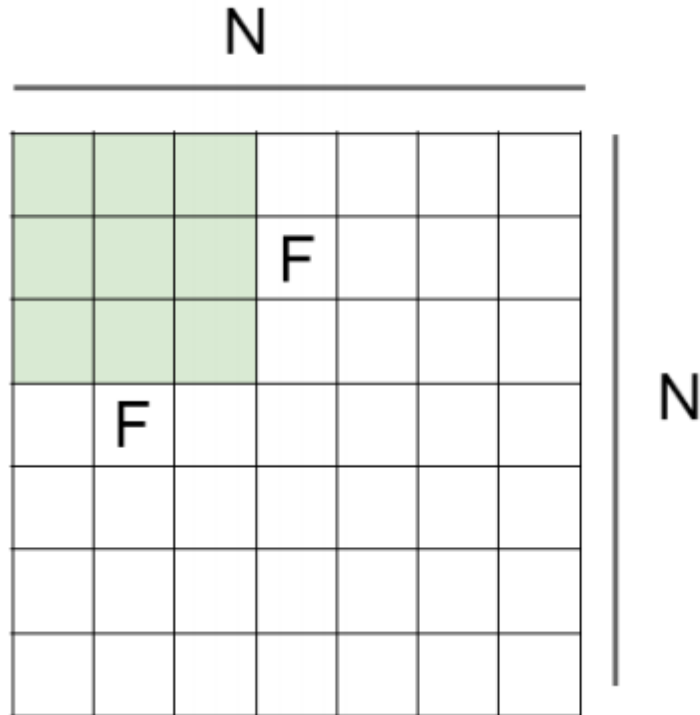
Spatial Dimensions – Closer Look



7x7 input (spatially)
assume 3x3 filter
applied **with stride 3?**

doesn't fit!
cannot apply 3x3 filter on
7x7 input with stride 3.

Output Size



Output size:

$$(N - F) / \text{stride} + 1$$

e.g. $N = 7, F = 3$:

$$\text{stride } 1 \Rightarrow (7 - 3) / 1 + 1 = 5$$

$$\text{stride } 2 \Rightarrow (7 - 3) / 2 + 1 = 3$$

$$\text{stride } 3 \Rightarrow (7 - 3) / 3 + 1 = 2.33 \therefore \backslash$$

Zero Padding

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$$(N - F) / \text{stride} + 1$$

7x7 output!

Zero Padding

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

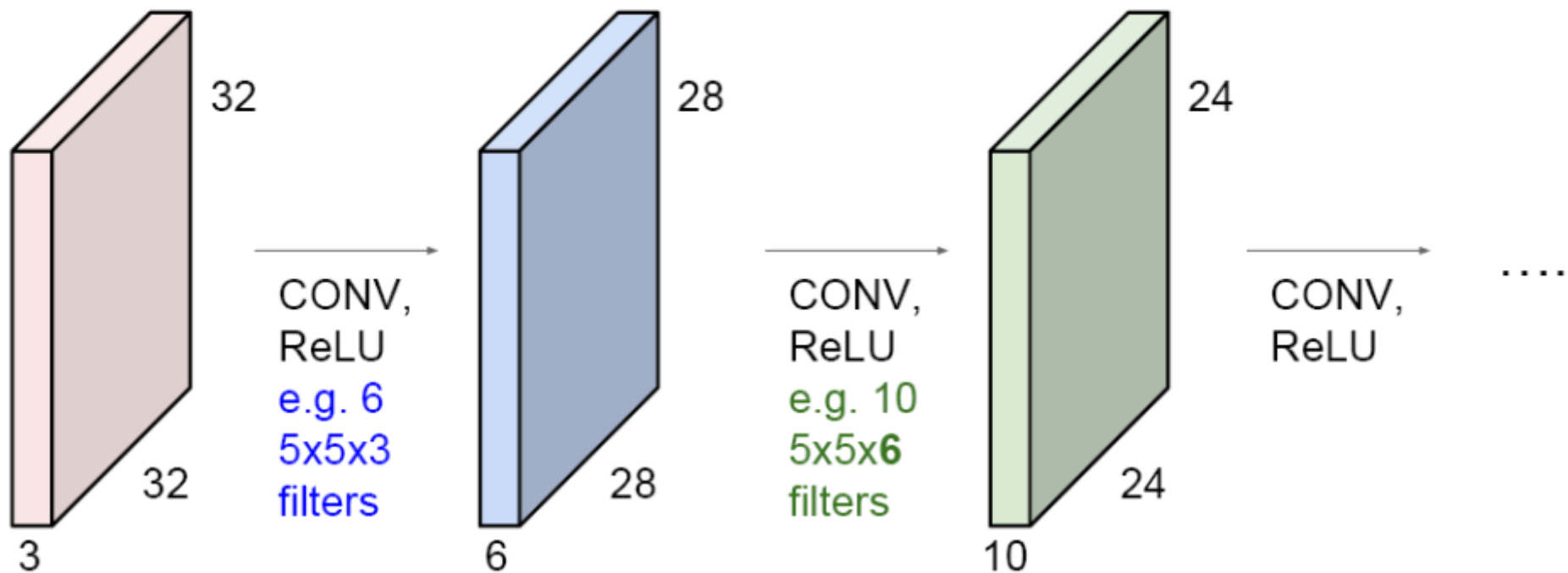
e.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

Zero Padding

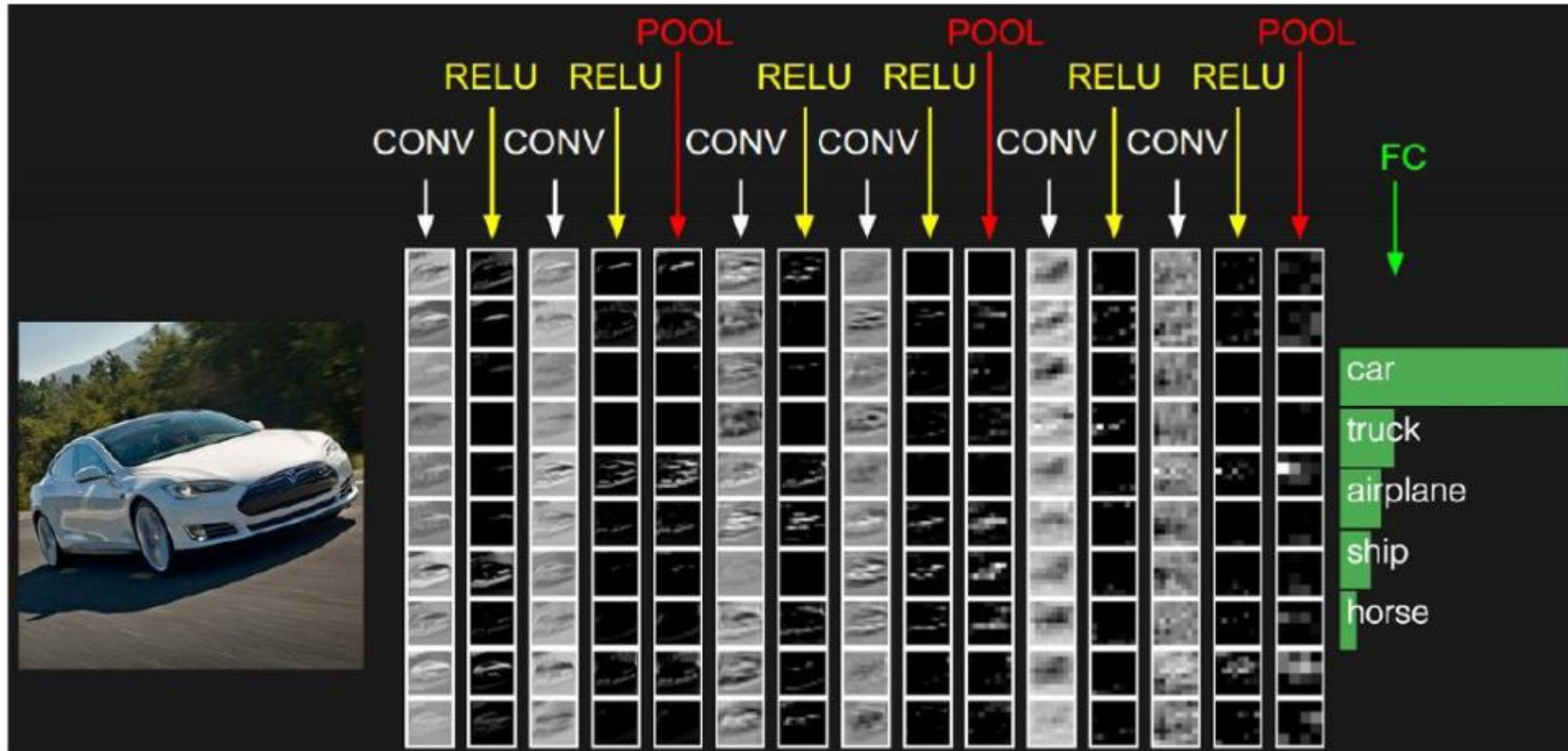
E.g. 32x32 input convolved repeatedly with 5x5 filters shrinks volumes spatially! (32 $\rightarrow$ 28 $\rightarrow$ 24 ...). Shrinking too fast is not good, doesn't work well.



Summary - Conv Layer

- Accepts a volume of size: $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - Number of filters: K
 - Spatial Extent of filters: F
 - Stride: S
 - Padding: P
- Produces a volume of size $W_2 \times H_2 \times D_2$ where
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$
 - $D_2 = K$
- With parameter sharing it gives $F \times F \times D_1$ weights per filter hence a total of $(F \times F \times D_1) \times K$ weights and K biases

Pooling and FC Layers



Pooling Operation

- Accepts a volume of size: $W_1 \times H_1 \times D_1$
- Requires the hyperparameters:
 - Spatial Extent: F
 - Stride: S
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$
- Introduces zero parameters

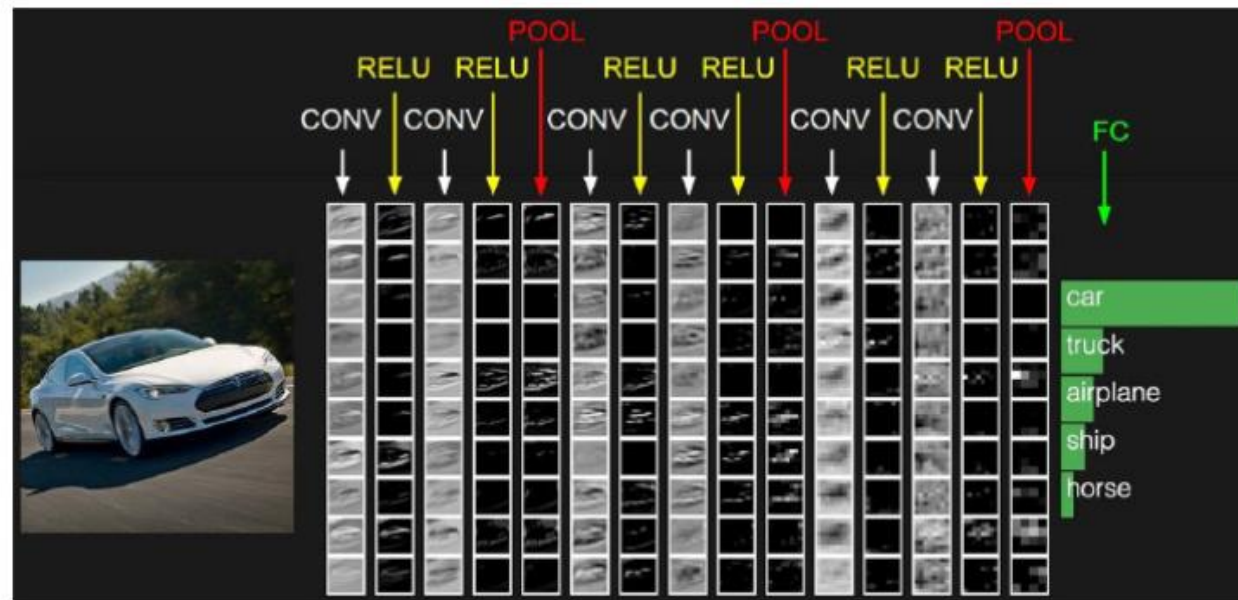
Common settings:

$F = 2, S = 2$

$F = 3, S = 2$

Fully Connected Layer (FC)

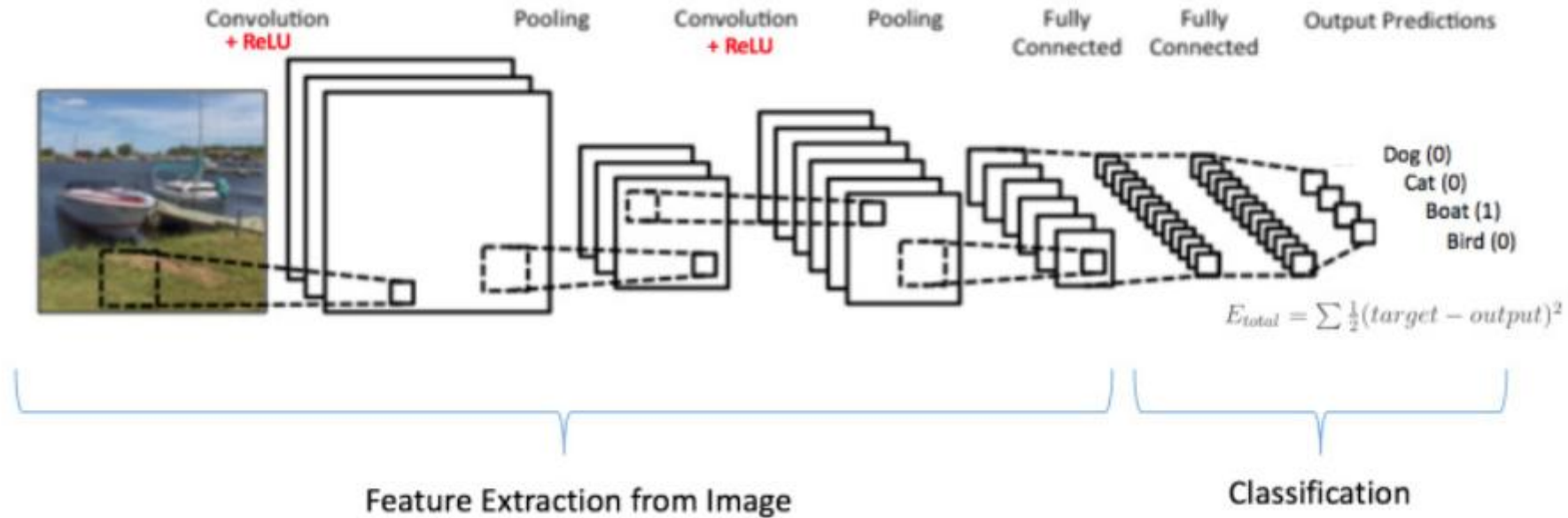
- Contains neurons that connect to the entire input volume, as in ordinary Neural Networks



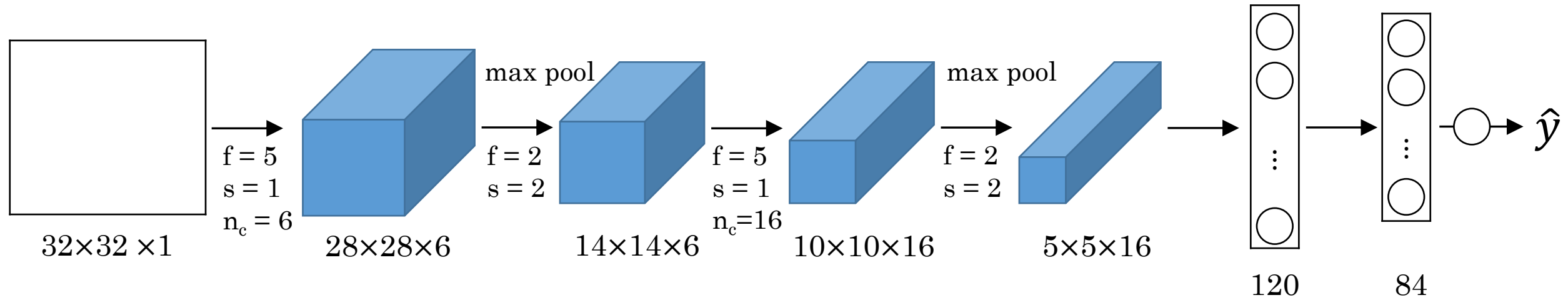
Fully Connected Layer (FC)

- This layer basically takes an input volume and outputs an N dimensional vector where N is the number of classes that the program has to choose from.
- For example, if we want a digit classification program, N would be 10 since there are 10 digits.
- Each number in this N dimensional vector represents the probability of a certain class.

Putting it all together



CNN Example



CNN Example

	Activation Shape	Activation Size	# of parameters
Input	(32,32,3)	3072	
CONV 1 (f=5,s=1,n _c = 8)	(28,28,8)	6272	$(5*5*3 + 1) * 8 = 608$
POOL 1	(14,14,8)	1568	
CONV 2 (f=5,s=1, n _c = 16)	(10,10,16)	1600	$(5*5*8 + 1) * 16 = 3216$
POOL 2	(5,5,16)	400	
FC 3	(120,1)	120	$400 * 120 + 120 = 48120$
FC 4	(84,1)	84	$120 * 84 + 84 = 10164$
Softmax	(10,1)	10	$84*10 + 10 = 850$

Intuition

- A CNN model can be thought as a combination of two components: feature extraction part and the classification part.
- The convolution + pooling layers perform feature extraction.
- For example given an image, the convolution layer detects features such as two eyes, long ears, four legs, a short tail and so on.
- The fully connected layers then act as a classifier on top of these features, and assign a probability for the input image being a dog.

Intuition

- The convolution layers learn such complex features by building on top of each other.
- The first layers detect edges, the next layers combine them to detect shapes, to following layers merge this information to infer that this is a nose.
- To be clear, the CNN doesn't know what a nose is. By seeing a lot of them in images, it learns to detect that as a feature.

Practice in Google Colab

- <https://shorturl.at/yfsu6>



Credit

- Machine Learning 1997 by T. Mitchell
- Deep Learning Specialization, Courses 1& 4 by Andrew Ng
 - deeplearning.ai